HOSTEL MANAGEMENT SYSTEM

A PROJECT REPORT

Submitted by

PRAJAN BALA S N

(2116241501144)

in partial fulfilment of the End Semester Examination for the course

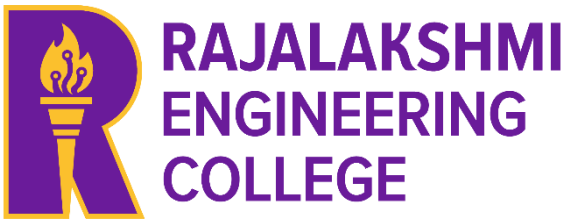
CB23F35 –

LOW-CODE APPLICATION DEVELOPMENT USING ZOHO CREATOR

in

BACHELOR OF TECHNOLOGY

ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING



RAJALAKSHMI ENGINEERING COLLEGE

(An Autonomous Institution, Affiliated to Anna University, Chennai)

Rajalakshmi Nagar, Thandalam, Tamil Nadu – 602 105

OCTOBER 2026

RAJALAKSHMI ENGINEERING COLLEGE

(An Autonomous Institution, Affiliated to Anna University, Chennai)

BONAFIDE CERTIFICATE

Certified that this project report titled "Hostel Management System" is the bonafide work of Prajan Bala S N (2116241501144), who carried out the individual project work on the Zoho Creator low-code platform for the course CB23F35 – Low-Code Application Development using Zoho Creator, during the academic year 2026–2027, Semester V, under my supervision.

SIGNATURE	**SIGNATURE**
Dr. M. Ayyadurai HEAD OF THE DEPARTMENT Department of Artificial Intelligence and Machine Learning Rajalakshmi Engineering College Thandalam – 602 105	Priya, M. E. COURSE FACULTY Assistant Professor Department of Artificial Intelligence and Machine Learning Rajalakshmi Engineering College

Submitted for the End Semester Examination held on _____.

INTERNAL EXAMINER	**EXTERNAL EXAMINER**

DECLARATION

I, Prajan Bala S N (2116241501144), declare that this report titled "Hostel Management System" documents the application in my Zoho Creator account for the course CB23F35 – Low-Code Application Development using Zoho Creator. The report identifies the application components, available evidence and remaining verification work.

The application was generated and deployed using the project's Zoho Creator CLI with AI-assisted development. AI assistance was also used to prepare this report and its browser evidence. These contributions and the platform documentation are acknowledged. I am responsible for reviewing the report, understanding the implementation and demonstrating the application. This declaration is to be signed only after my review.

Place: Chennai
Date: _____

SIGNATURE OF THE STUDENT
Prajan Bala S N
2116241501144

ACKNOWLEDGEMENT

I thank the management of Rajalakshmi Engineering College for providing the institutional facilities and learning environment for this project.

I thank the Principal of Rajalakshmi Engineering College for supporting practical learning and application development.

I thank Dr. M. Ayyadurai, Head of the Department of Artificial Intelligence and Machine Learning, for the academic support provided during the course.

I thank Priya, M. E., Assistant Professor and course faculty, for guidance in the course and the project-report requirements.

I acknowledge Zoho Corporation for the Creator platform and official documentation. I also acknowledge the local Zoho Creator CLI, AI-assisted application development and report preparation, and Playwright for browser inspection and screenshot capture. I thank my family and friends for their support.

Prajan Bala S N

ABSTRACT

Hostel administration requires consistent information about residents, rooms, accommodation charges, leave and maintenance. When these activities are maintained in separate registers or spreadsheets, staff must repeatedly enter student details, check room availability manually and reconcile payments with room records. The Hostel Management System addresses this problem through a Zoho Creator application for the administrator, warden, accountant and student. The application contains six related forms: Rooms, Students, Room Allocations, Fee Payments, Leave Requests and Complaints. Lookup fields connect residents and rooms, while the Fee Items subform stores multiple charges within a bill. The source configuration contains 29 form workflows, two reusable Deluge functions and one daily overdue schedule. These scripts define occupancy updates, duplicate and capacity checks, rent retrieval, item totals, inclusive leave duration, approval decisions and email notifications. Fifteen reports provide eleven lists, three Kanban boards and one summary view. A dashboard adds six operational counters, four charts, four action buttons and two embedded reports. The live application was inspected on 20 October 2026 through an authenticated browser using Playwright, and its records were read through the existing CLI client. The verified dataset contains 12 rooms, 15 students, 14 allocations, 20 fee records, 11 leave requests and 11 complaints. Twenty read-only checks passed, including consistency between active allocations, occupied beds and current-room lookups. The snapshot contains 12 active allocations, seven unpaid bills totalling INR 70442, four pending leave requests and six unresolved complaints. These results establish the current data state; they do not establish successful execution of every workflow or delivery of notification emails. The dashboard and its charts were verified in the live app. External business-service integration is outside the current scope; role-specific sign-in tests and user acceptance feedback remain to be recorded. The project demonstrates how linked low-code forms and Deluge rules can centralise hostel operations, while identifying the remaining work needed before a complete assessed demonstration.

Keywords: Zoho Creator; Deluge; Hostel Management; Room Allocation; Workflow Automation; Role Based Access

PROJECT AT A GLANCE

Attribute	Details
Project Title	Hostel Management System
Application Domain	Education and hostel administration
Student Name and Register Number	Prajan Bala S N — 2116241501144
Zoho Creator Edition	Trial account observed; paid edition not confirmed
Application Name and Link Name	Hostel Management System / hostel-management-system
Authenticated Live URL	https://creatorapp.zoho.in/prajanbala.sn.2024.aiml_r892/hostel-management-system/
Demo access	Use assigned Zoho accounts; credentials excluded
Forms and Subforms	6 main forms / 1 embedded Fee Items subform
Reports by Type	11 List / 3 Kanban / 1 Summary
Pages and Dashboards	1 Dashboard / 6 counters / 4 charts / 4 action buttons / 2 embedded reports
Workflows	29 form workflows / 1 daily schedule; scripted leave decision
Deluge Functions and Custom Actions	2 functions / no separate custom report action
Integrations	Creator REST API through CLI; Deluge sendmail defined
Roles and Profiles	Administrator and three custom permission profiles; role hierarchy and test users defined in source
Environments Used	Production target only; Stage and Development not verified
Deployment and Verification Date	20 October 2026

TABLE OF CONTENTS

ABSTRACT iv
PROJECT AT A GLANCE v
LIST OF TABLES ix
LIST OF FIGURES x
LIST OF ABBREVIATIONS xi
INTRODUCTION 1
1.1 Background 1
1.2 Problem Statement 1
1.3 Objectives 1
1.4 Scope of the Project 2
1.5 Justification for a Low-Code Approach 2
1.6 Organisation of the Report 2
LITERATURE SURVEY AND SYSTEM STUDY 3
2.1 Study of the Existing System 3
2.2 Limitations of the Existing System 3
2.3 Low-Code versus Traditional Development 3
2.4 Literature Survey 4
2.5 Emerging Trends in Low-Code Development 4
2.6 Proposed System 5
REQUIREMENT ANALYSIS 6
3.1 Stakeholders and User Roles 6
3.2 Functional Requirements 6
3.3 Non-Functional Requirements 7
3.4 Hardware and Software Requirements 7
3.5 Use Case Model 8
SYSTEM DESIGN 9
4.1 System Architecture 9
4.2 Data Model 9
4.3 Form Design 11
4.4 Process and Workflow Design 15
4.5 User Interface and Navigation Design 16
4.6 Access Control Design 16
IMPLEMENTATION 17
5.1 Application Setup 17
5.2 Forms and Data Capture 17

5.3 Reports 21
5.4 Pages and Dashboards 24
5.5 Deluge Scripting 25
5.6 Workflows and Automation 26
5.7 Integrations 29
5.8 User Management and Role-Based Access 30
5.9 AI and Analytics Features 31
5.10 Pro-Code Extension 32
5.11 Mobile and Portal Access 32
TESTING AND DEBUGGING 33
6.1 Testing Strategy 33
6.2 Test Cases 33
6.3 Debugging Deluge Scripts 35
6.4 Defect Log 36
6.5 User Acceptance Testing 37
DEPLOYMENT, SECURITY AND GOVERNANCE 38
7.1 Environments and Deployment Strategy 38
7.2 Performance Optimisation 39
7.3 Security Features 39
7.4 Governance and Administration 40
RESULTS AND DISCUSSION 41
8.1 Application Output 41
8.2 Achievement of Objectives 42
8.3 Comparison with the Existing System 42
8.4 Limitations of the Low-Code Approach 43
8.5 Reflection on the No-Code → Low-Code → Pro-Code Spectrum 43
CONCLUSION AND FUTURE ENHANCEMENTS 44
9.1 Conclusion 44
9.2 Future Enhancements 44
9.3 Learning Outcomes and Career Relevance 44
REFERENCES 45
APPENDIX A – ZOHO CREATOR FEATURE COVERAGE CHECKLIST 46
APPENDIX B – COURSE OUTCOME MAPPING 49
APPENDIX C – COMPLETE DELUGE SOURCE CODE 50
C.2 DS-01 pendingDues 50
C.3 DS-02 refreshRoomOccupancy 50
C.4 DS-03 Allocation_Added 50

C.5 DS-04 Allocation_Deleted 51
C.6 DS-05 Allocation_Form_Load 51
C.7 DS-06 Allocation_Room_Selected 52
C.8 DS-07 Allocation_Status_Changed 52
C.9 DS-08 Allocation_Updated 53
C.10 DS-09 Allocation_Validate 53
C.11 DS-10 Complaint_Form_Load 54
C.12 DS-11 Complaint_Raised 55
C.13 DS-12 Complaint_Resolved 55
C.14 DS-13 Complaint_Status_Changed 56
C.15 DS-14 Complaint_Student_Selected 56
C.16 DS-15 Complaint_Validate 56
C.17 DS-16 Fee_Form_Load 57
C.18 DS-17 Fee_Mode_Changed 58
C.19 DS-18 Fee_Receipt 58
C.20 DS-19 Fee_Status_Changed 58
C.21 DS-20 Fee_Student_Selected 59
C.22 DS-21 Fee_Validate 60
C.23 DS-22 Leave_Dates_Changed 61
C.24 DS-23 Leave_Decision 61
C.25 DS-24 Leave_Form_Load 62
C.26 DS-25 Leave_Review_Load 62
C.27 DS-26 Leave_Validate 62
C.28 DS-27 Room_Form_Load 63
C.29 DS-28 Room_Type_Changed 64
C.30 DS-29 Room_Validate 64
C.31 DS-30 Student_Form_Load 65
C.32 DS-31 Student_Validate 65
C.33 DS-32 Daily_Fee_Overdue_Check 67
APPENDIX D – USER MANUAL 68
D.1 Accessing the Application 68
D.2 Instructions for Administrator and Warden 68
D.3 Instructions for Accountant and Student 68
APPENDIX E – EVALUATION RUBRIC 69
APPENDIX F – VIVA-VOCE RECORD SHEET 71

LIST OF TABLES

Table 2.1: Comparison of Low-Code and Traditional Development for the Proposed Application....3
Table 2.2: Summary of Literature Survey....4
Table 3.1: Stakeholders and User Roles....6
Table 3.2: Functional Requirements....6
Table 3.3: Non-Functional Requirements....7
Table 3.4: Hardware and Software Requirements....7
Table 3.5: Use Case Description for Room Allocation....8
Table 4.1: Entities and Relationships....10
Table 4.2: Field Specification for Rooms....11
Table 4.3: Field Specification for Students....11
Table 4.4: Field Specification for Room Allocations....12
Table 4.5: Field Specification for Fee Payments....13
Table 4.6: Fee Items Subform Fields....14
Table 4.7: Field Specification for Leave Requests....14
Table 4.8: Field Specification for Complaints....14
Table 4.9: Role Permission Matrix....16
Table 5.1: Reports Implemented....21
Table 5.2: Deluge Scripts Implemented....25
Table 5.3: Workflows Implemented....26
Table 5.4: Integrations Implemented....29
Table 6.1: Test Cases....33
Table 6.2: Defect Log....36
Table 6.3: User Acceptance Feedback....37
Table 7.1: Deployment Checklist....38
Table 7.2: Performance Optimisation Results....39
Table 7.3: Security Controls Implemented....39
Table 8.1: Mapping of Objectives to Outcomes....42
Table 8.2: Existing System versus Proposed Application....42
Table A.1: Feature Coverage Checklist....46
Table B.1: Mapping of Course Outcomes to Report Chapters....49
Table E.1: Evaluation Rubric for the Individual Project....69
Table E.2: Performance Level Descriptors....70
Table F.1: Viva-Voce Questions and Responses....71
Table F.2: Marks Summary....71

LIST OF FIGURES

Figure 2.1: Illustrative Manual Hostel Process 3
Figure 3.1: Hostel Management Use Cases and Intended Actors 8
Figure 4.1: Hostel Management System Architecture 9
Figure 4.2: Entity Relationships in the Hostel Data Model 10
Figure 4.3: Allocation Validation and Occupancy Workflow 15
Figure 4.4: Application Navigation Structure 16
Figure 5.1: Live Application Home and Room Inventory 17
Figure 5.2: Students Form in the Creator Builder 18
Figure 5.3: Students Form in Live Mode 18
Figure 5.4: Rooms Form in Live Mode 19
Figure 5.5: Room Allocations Form in Live Mode 19
Figure 5.6: Fee Payments Form in Live Mode 20
Figure 5.7: Leave Requests Form in Live Mode 20
Figure 5.8: Complaints Form in Live Mode 21
Figure 5.9: Rooms Grouped by Status in a Kanban Report 22
Figure 5.10: Leave Requests Grouped by Approval Status 23
Figure 5.11: Complaints Grouped by Status 23
Figure 5.12: Active Student Directory Summary Report 24
Figure 5.13: Live Dashboard Counters Quick Actions and Charts 25
Figure 5.14: Form Workflow Definitions in the Creator Builder 29
Figure 5.15: Live Pending and Overdue Fee Records Verified through the CLI 30
Figure 5.16: Application Settings with Permission Entry Points 31
Figure 5.17: Dashboard Analytics and Embedded Operational Reports 32
Figure 6.1: Imported Components Listed in the Application IDE 36
Figure 7.1: Populated Student Report in the Authenticated Live Application 38
Figure 8.1: Existing Active Allocation for Meera Nair in C102 41
Figure 8.2: Existing October Fee for Meera Nair Marked Pending 41
Figure 8.3: Existing Open Cleaning Complaint for Meera Nair in C102 42

LIST OF ABBREVIATIONS

Abbreviation	Expansion
API	Application Programming Interface
CLI	Command Line Interface
CRUD	Create Read Update Delete
ER	Entity Relationship
FR	Functional Requirement
INR	Indian Rupee
JSON	JavaScript Object Notation
LCAP	Low Code Application Platform
NFR	Non Functional Requirement
OAuth	Open Authorization
RBAC	Role Based Access Control
REST	Representational State Transfer
UAT	User Acceptance Testing
UI	User Interface

CHAPTER 1
INTRODUCTION

1.1 Background

This project addresses hostel administration in a college setting. Wardens maintain accommodation and resident records, accountants track monthly charges, and students submit leave and maintenance requests. The implementation is an educational demonstration with realistic sample records, not a claim of operation by a particular hostel. Its live dataset spans three blocks, 12 rooms and 15 student records.

1.2 Problem Statement

Hostel staff need one consistent record of who occupies each bed, what each resident owes, and which leave or maintenance requests require action. Maintaining these facts in separate registers creates a risk of conflicting room assignments, repeated entry and incomplete follow-up.

- Room availability must be reconciled with active accommodation allocations.
- Monthly bills and payment status must remain linked to the correct resident and room.
- Leave decisions and unresolved complaints need a visible status and an accountable reviewer.

1.3 Objectives

- Maintain six related forms and avoid repeating student and room information through lookup fields.
- Define checks for duplicate allocations, room capacity, invalid dates and repeated monthly bills.
- Provide fifteen useful reports for residents, room availability, fees, leave and maintenance.
- Automate occupancy, fee totals, leave duration and notifications through Deluge workflows.
- Define Administrator, Warden, Accountant and Student access rules, then distinguish configured rules from verified role access.

1.4 Scope of the Project

The implemented scope comprises six browser forms, related records, a Fee Items subform, fifteen reports, one dashboard and source-defined Deluge automation. The dashboard contains six counters, four charts, four action buttons and two embedded reports. The demonstrated session uses the owner account. Payment-gateway integration, an external CRM connector, customer portal, native mobile demonstration and cross-environment promotion remain outside the verified scope.

1.5 Justification for a Low-Code Approach

The hostel processes map naturally to structured forms, related records and event-driven rules. Model-based low-code development can express these structures at a higher level of abstraction than a fully custom application [1]. Creator supplies the form, report and workflow environment used here [2]. The CLI retains readable definitions, while Deluge expresses rules that exceed simple field configuration. No development-time or cost saving is claimed without a measured baseline.

1.6 Organisation of the Report

Chapter 2 reviews the existing system and related low-code literature. Chapter 3 presents the requirement analysis. Chapter 4 describes the system design. Chapter 5 details the implementation on Zoho Creator. Chapter 6 covers testing and debugging. Chapter 7 discusses deployment, performance optimisation, security and governance. Chapter 8 presents results and discussion, and Chapter 9 concludes the report with future enhancements.

CHAPTER 2
LITERATURE SURVEY AND SYSTEM STUDY

2.1 Study of the Existing System

The baseline is an illustrative manual hostel process: a student submits details or a request; an office member checks the room register or fee ledger; the warden or accountant takes action; and the result is copied into separate records. No survey of a named hostel was performed, so the baseline describes the business problem rather than measured institutional practice.

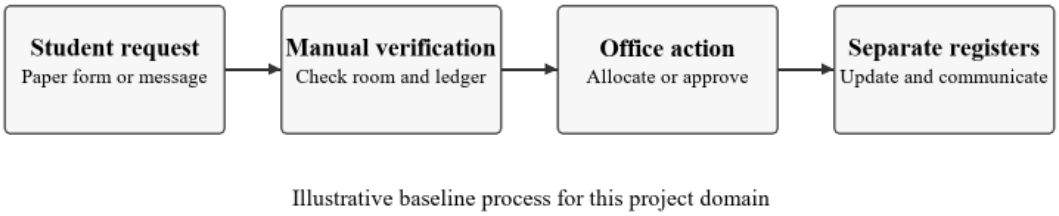


Figure 2.1: Illustrative Manual Hostel Process

2.2 Limitations of the Existing System

- Repeated entry of student names and room details can create inconsistent records.
- Separate ledgers make a combined view of accommodation and dues difficult.
- Paper or message-based approvals provide limited visibility of pending work.

2.3 Low-Code versus Traditional Development

Table 2.1: Comparison of Low-Code and Traditional Development for the Proposed Application

Criterion	Traditional Development	Low Code on Zoho Creator
Development effort	Custom UI, database and server required	Forms, reports and Deluge configured; no measured effort comparison
Skills	Application, database and deployment code	Form modelling, Deluge and CLI knowledge
Hosting	Provision and maintain infrastructure	Creator-hosted authenticated application
Mobile	Build and verify a mobile interface	Native Creator mobile access not tested

Criterion	Traditional Development	Low Code on Zoho Creator
Access control	Implement and test authorisation	Profiles and filtered reports; role tests pending
Maintenance	Coordinate code and schema changes	Retain YAML and Deluge definitions with exports
Customisation	Broad control with greater implementation effort	Builder and API boundaries constrain some changes
Cost	No cost model measured	Trial account observed; no cost-saving claim

2.4 Literature Survey

Table 2.2: Summary of Literature Survey

No.	Source	Title	Finding	Project relevance
1	Bucaioni et al. 2022 [1]	Modelling in low-code development	Reviews low-code foundations and model-driven relationships	Supports model-based forms and workflow design
2	Zoho [2]	Form workflows	Events can trigger configured actions	Used to organise source workflow triggers
3	Zoho [3]	Scripting for Subforms	Child rows can be populated and validated	Fee Items stores multiple charges
4	Zoho [4]	For each record	Criteria fetch and record iteration	Allocation checks and overdue processing
5	Zoho [5]	User permissions	Module and record access need explicit control	Supports the proposed permission matrix
6	Zoho [6]	Get Records API v2.1	Reports expose records through authenticated API	Live metadata and record verification

2.5 Emerging Trends in Low-Code Development

AI-assisted generation can accelerate an initial definition, but every generated rule still needs review against the domain. This project used AI-assisted development and a CLI to produce repeatable application definitions. Citizen-developer maintainability comes from clear field names and reports; higher-risk allocation and fee rules remain explicit Deluge code. No Zia prediction, OCR or machine-learning model is deployed.

2.6 Proposed System

The solution stores room inventory, resident details, allocations, fees, leave and complaints in related Creator forms. Filtered reports support daily tasks, and Kanban boards organise status-based work. The Dashboard now combines operational counters, charts, quick actions and pending-work reports. Deluge scripts define validation and updates across the related records.

CHAPTER 3
REQUIREMENT ANALYSIS

3.1 Stakeholders and User Roles

Table 3.1: Stakeholders and User Roles

Role	Description	Responsibilities
Administrator	Application owner	Manage all modules, definitions and access
Warden	Hostel operations	Manage rooms, residents and allocations; review leave and complaints
Accountant	Fee administration	Prepare and settle bills; read student and room references
Student	Hostel resident	View available rooms; submit and read own leave and complaints

3.2 Functional Requirements

Table 3.2: Functional Requirements

FR ID	Requirement	Roles	Priority
FR-01	Register students with unique student ID and email	Administrator, Warden	High
FR-02	Maintain capacity, rent and room status	Administrator, Warden	High
FR-03	Allocate and vacate rooms without duplicate active stays or overcapacity	Administrator, Warden	High
FR-04	Prepare itemised monthly bills and record payment details	Administrator, Accountant	High
FR-05	Submit leave and record approval or rejection with remarks	Student, Warden	High
FR-06	Submit, track and resolve room complaints	Student, Warden	High
FR-07	Provide list, summary and Kanban operational reports	Permitted profiles	High
FR-08	Define notification and overdue processing rules	Administrator	Medium
FR-09	Restrict module actions and own-record visibility by profile	All roles	High

FR ID	Requirement	Roles	Priority
FR-10	Provide an operational overview dashboard	Administrator, Warden	High

3.3 Non-Functional Requirements

Table 3.3: Non-Functional Requirements

NFR ID	Category	Requirement and verification status
NFR-01	Performance	Operational reports should remain usable at expected volume; benchmark not measured
NFR-02	Security	Users must access only permitted modules and records; separate-role tests pending
NFR-03	Usability	Consistent browser navigation and clear required fields; browser observed
NFR-04	Availability	Authenticated hosted access required; only this inspection session verified
NFR-05	Maintainability	Retain readable YAML, Deluge source and repeatable exports
NFR-06	Auditability	Retain change history and test evidence; native audit event not verified

3.4 Hardware and Software Requirements

Table 3.4: Hardware and Software Requirements

Category	Specification
Platform	Zoho Creator India data centre; trial account observed
Application scripts	Deluge
CLI	TypeScript on Bun; YAML definitions; Playwright browser driver
Browser inspection	Authenticated Chromium with Playwright [7]
API verification	ExistingCreatorClient using OAuth and Creator REST API [6]
Other business services	No CRM, payment gateway or Analytics connector implemented
Mobile	Native mobile application and portal not verified
Client environment	Windows workstation and internet connection; no minimum hardware benchmark

3.5 Use Case Model

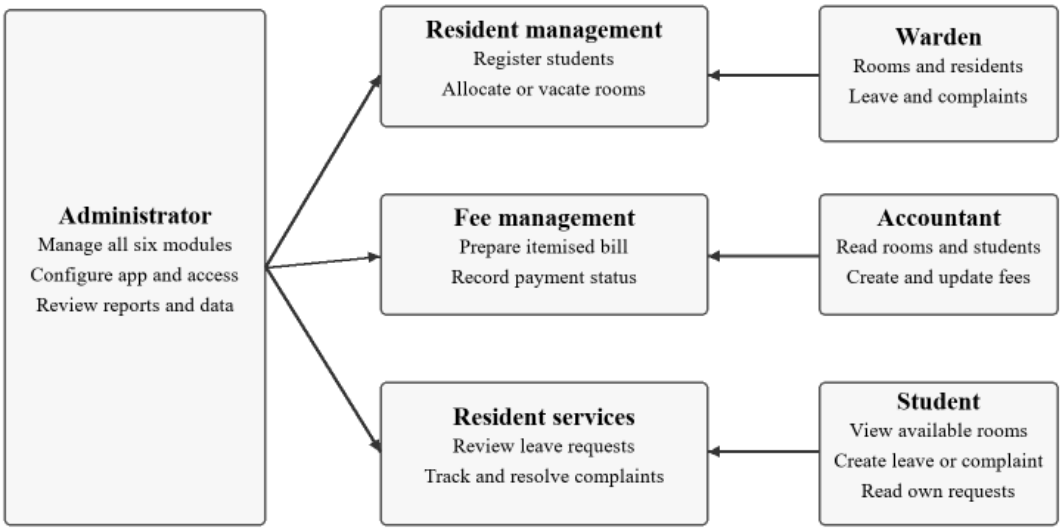


Figure 3.1: Hostel Management Use Cases and Intended Actors

Table 3.5: Use Case Description for Room Allocation

Use Case ID	UC-01
Use Case Name	Allocate a resident to a room
Actors	Administrator or Warden
Precondition	Active student; room not under maintenance; at least one free bed
Main Flow	Select student and room; check dates and status; validate; save; recount occupancy; update current room
Alternative Flow	Reject duplicate active stay, full room or invalid dates
Postcondition	Allocation recorded; room occupancy and student lookup should agree
Related Requirements	FR-01, FR-02, FR-03, FR-08

CHAPTER 4
SYSTEM DESIGN

4.1 System Architecture

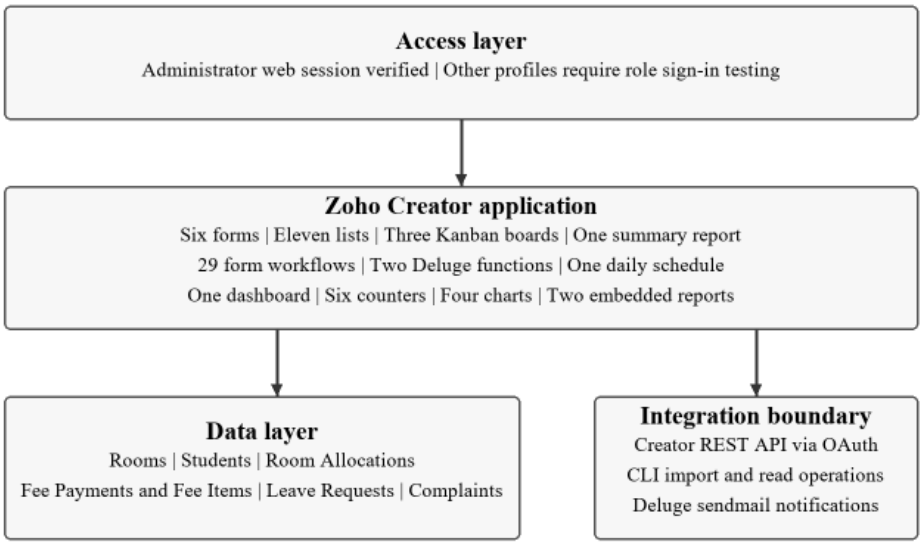


Figure 4.1: Hostel Management System Architecture

The browser presents the six modules and their reports. Creator holds the data and executes configured Deluge events. The existing CLI uses OAuth-authenticated Creator APIs for metadata and record access and Playwright for builder operations that require the browser. Notification code uses sendmail; no external business-service connector is configured. The Fee Items subform is a child structure within Fee Payments [3].

4.2 Data Model

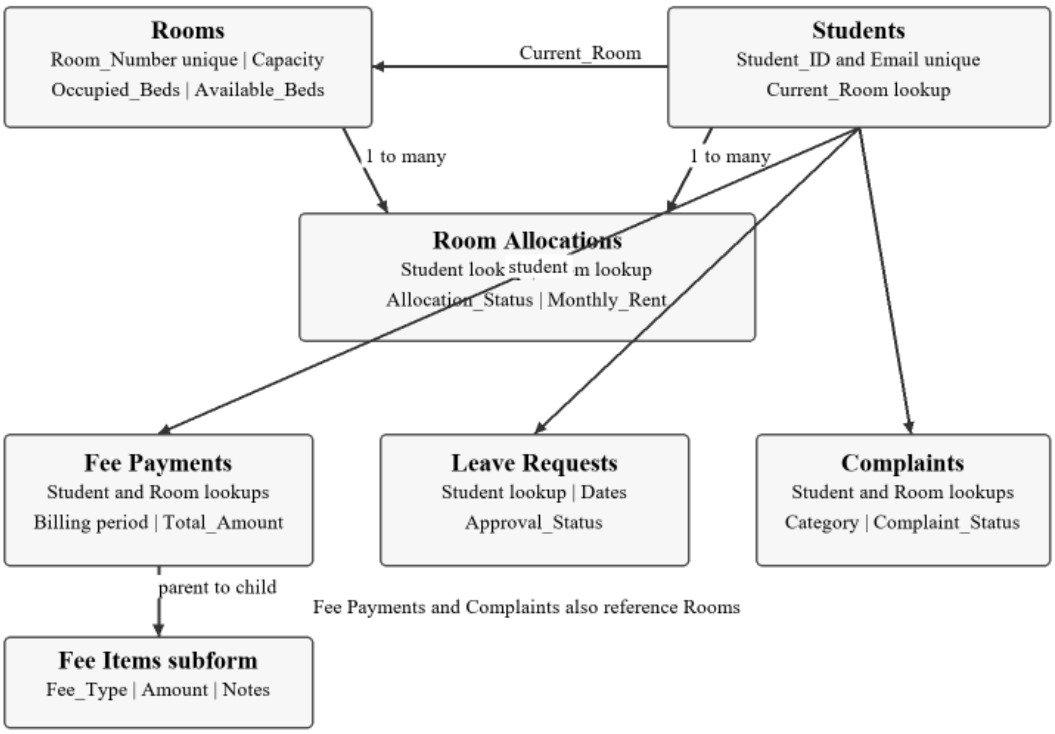


Figure 4.2: Entity Relationships in the Hostel Data Model

Table 4.1: Entities and Relationships

Entity	Key fields	Related entity	Relationship	Implementation
Rooms	Room_Number unique	Room Allocations	One to many	Room lookup in allocations
Students	Student_ID and Email unique	Allocations, Fees, Leave, Complaints	One to many	Student lookups
Room Allocations	Allocation_No	Rooms and Students	Many to one	Two lookups
Fee Payments	Receipt_No, billing period	Fee Items	Parent and child	Embedded subform
Students	Current_Room	Rooms	Many to one	Current room lookup
Complaints	Complaint_No	Students and Rooms	Many to one	Two lookups

4.3 Form Design

Table 4.2: Field Specification for Rooms

Field	Link name	Type	Required	Rule or default	Choices or lookup
Room Number	Room_Number	Single Line	Yes	Unique room identifier	—
Block	Block	Drop Down	Yes	Mandatory	A Block, B Block, C Block
Floor	Floor	Number	Yes	0 to 10	—
Room Type	Room_Type	Drop Down	Yes	Mandatory	Single, Double, Triple
Capacity (beds)	Capacity	Number	Yes	1 to 4; not below occupancy	—
Occupied Beds	Occupied_Beds	Number	No	Derived from active allocations	—
Available Beds	Available_Beds	Number	No	Capacity less occupied	—
Monthly Rent	Monthly_Rent	Currency	Yes	INR 1000 to 50000	—
Amenities	Amenities	Checkbox	No	Optional	Attached Bathroom, Air Conditioning, Wi-Fi, Study Table, Balcony
Room Status	Room_Status	Drop Down	Yes	Maintenance preserved; otherwise capacity based	Available, Full, Under Maintenance

Table 4.3: Field Specification for Students

Field	Link name	Type	Required	Rule or default	Choices or lookup
Student ID	Student_ID	Single Line	Yes	Unique; starts HS	—
Full Name	Full_Name	Single Line	Yes	Mandatory	—
Gender	Gender	Radio	Yes	Mandatory	Male, Female, Other
Date of Birth	Date_of_Birth	Date	Yes	Past date; age 16 to 35	—
Email	Email	Email	Yes	Unique	—
Phone	Phone	Phone	Yes	At least 10 digits; duplicate check	—

Field	Link name	Type	Required	Rule or default	Choices or lookup
Department	Department	Drop Down	Yes	Mandatory	AI & DS, CSE, ECE, EEE, IT, MECH, CIVIL
Year of Study	Year_of_Study	Drop Down	Yes	Mandatory	I Year, II Year, III Year, IV Year
Guardian Name	Guardian_Name	Single Line	Yes	Mandatory	—
Guardian Phone	Guardian_Phone	Phone	Yes	At least 10 digits; differs from student	—
Permanent Address	Permanent_Address	Multi Line	No	Optional	—
ID Proof (Aadhaar / College ID)	ID_Proof	File Upload	No	Optional	—
Current Room	Current_Room	Lookup	No	Updated by allocation workflows	Rooms
Student Status	Student_Status	Drop Down	Yes	Active default in source	Active, Checked Out

Table 4.4: Field Specification for Room Allocations

Field	Link name	Type	Required	Rule or default	Choices or lookup
Allocation No	Allocation_No	Auto Number	No	Optional	—
Student	Student	Lookup	Yes	Active; one active stay	Students
Room	Room	Lookup	Yes	Free bed; not under maintenance	Rooms
Allocation Date	Allocation_Date	Date	Yes	Today default in source	—
Expected Vacate Date	Expected_Vacate_Date	Date	No	After allocation date	—
Monthly Rent	Monthly_Rent	Currency	No	Fetched from room	—
Security Deposit	Security_Deposit	Currency	No	INR 5000 default in source	—
Allocation Status	Allocation_Status	Drop Down	Yes	Mandatory	Active, Vacated

Field	Link name	Type	Required	Rule or default	Choices or lookup
Vacated On	Vacated_On	Date	No	Required for Vacated status	—
Remarks	Remarks	Multi Line	No	Optional	—

Table 4.5: Field Specification for Fee Payments

Field	Link name	Type	Required	Rule or default	Choices or lookup
Receipt No	Receipt_No	Auto Number	No	Optional	—
Student	Student	Lookup	Yes	Room and rent item fetched	Students
Room	Room	Lookup	No	Optional	Rooms
Billing Month	Billing_Month	Drop Down	Yes	Mandatory	January, February, March, April, May, June, July, August, September, October, November, December
Billing Year	Billing_Year	Number	Yes	Validated year	—
Fee Items	Fee_Items	Subform	No	At least one positive charge	Fee Type, Amount, Notes
Total Amount	Total_Amount	Currency	No	Sum of Fee Items amounts	—
Due Date	Due_Date	Date	Yes	Today plus 10 days in source	—
Payment Status	Payment_Status	Drop Down	Yes	Mandatory	Pending, Paid, Overdue
Payment Date	Payment_Date	Date	No	Required when Paid; no future date	—
Payment Mode	Payment_Mode	Drop Down	No	Required when Paid	Cash, UPI, Card, Bank Transfer
Transaction Reference (UPI / card / bank)	Transaction_Reference	Single Line	No	Required for UPI, Card or Bank Transfer	—

Table 4.6: Fee Items Subform Fields

Field	Link name	Type	Required	Rule	Choices
Fee Type	Fee_Type	Drop Down	Yes	Classify charge	Rent, Mess, Electricity, Maintenance, Late Fine, Other
Amount	Amount	Currency	Yes	Greater than zero	INR
Notes	Notes	Single Line	No	Optional explanation	—

Table 4.7: Field Specification for Leave Requests

Field	Link name	Type	Required	Rule or default	Choices or lookup
Student	Student	Lookup	Yes	Active resident required	Students
Leave Type	Leave_Type	Drop Down	Yes	Mandatory	Home Visit, Medical, Personal, Academic Event
From Date	From_Date	Date	Yes	Tomorrow default in source	—
To Date	To_Date	Date	Yes	Not before From Date; at most 30 days	—
Number of Days	Number_of_Days	Number	No	Inclusive date difference	—
Reason	Reason	Multi Line	Yes	Mandatory	—
Approval Status	Approval_Status	Drop Down	Yes	Pending default; rejection needs remarks	Pending, Approved, Rejected
Warden Remarks	Warden_Remarks	Multi Line	No	Required on rejection	—

Table 4.8: Field Specification for Complaints

Field	Link name	Type	Required	Rule or default	Choices or lookup
Complaint No	Complaint_No	Auto Number	No	Optional	—
Student	Student	Lookup	Yes	Current room fetched	Students
Room	Room	Lookup	No	Optional	Rooms

Field	Link name	Type	Required	Rule or default	Choices or lookup
Category	Category	Drop Down	Yes	Mandatory	Electrical, Plumbing, Furniture, Cleaning, Internet, Other
Priority	Priority	Radio	Yes	Medium default in source	Low, Medium, High
Description	Description	Multi Line	Yes	Mandatory	—
Photo / Attachment	Attachment	File Upload	No	Optional	—
Raised On	Raised_On	Date	Yes	Today default; not future	—
Complaint Status	Complaint_Status	Drop Down	Yes	Open default; one unresolved room/category	Open, In Progress, Resolved
Resolved On	Resolved_On	Date	No	Required when resolved; not before raised	—
Resolution Notes	Resolution_Notes	Multi Line	No	Required when resolved	—

4.4 Process and Workflow Design

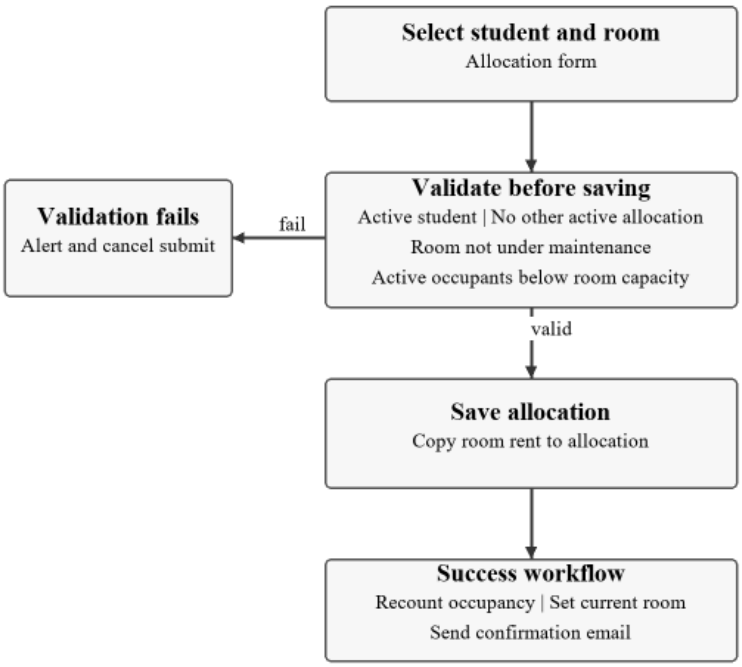


Figure 4.3: Allocation Validation and Occupancy Workflow

4.5 User Interface and Navigation Design

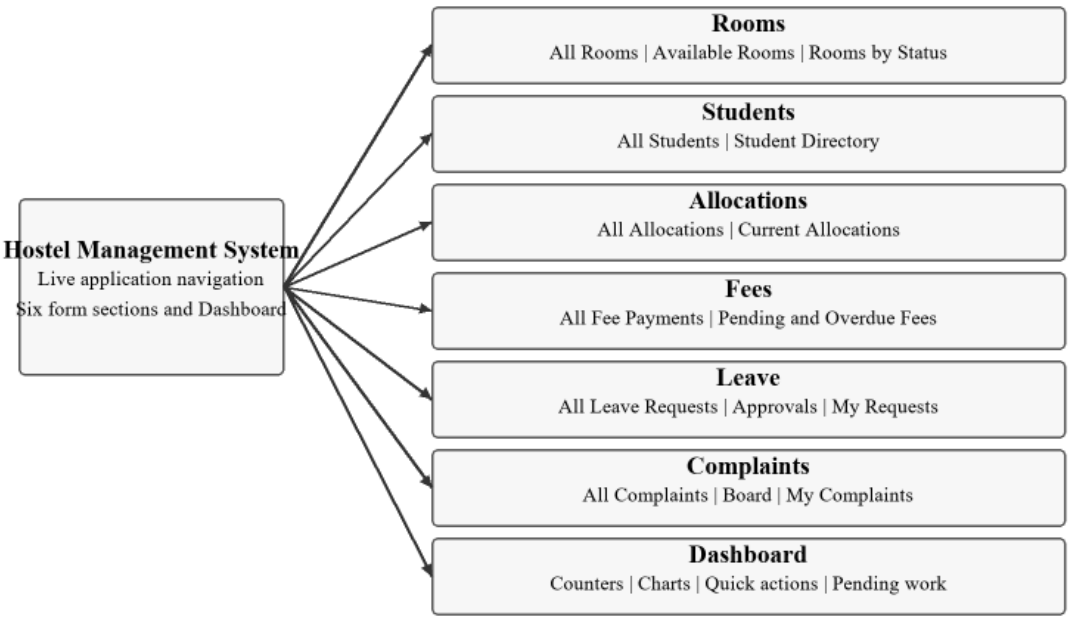


Figure 4.4: Application Navigation Structure

4.6 Access Control Design

Table 4.9: Role Permission Matrix

Component	Administrator	Warden	Accountant	Student
Rooms	A E V D	A E V	V	V Available
Students	A E V D	A E V	V and export	—
Room Allocations	A E V D	A E V	—	—
Fee Payments	A E V D	V	A E V	—
Leave Requests	A E V D	E V	—	A and V own
Complaints	A E V D	E V	—	A and V own
Dashboard	V	V	—	—

CHAPTER 5
IMPLEMENTATION

5.1 Application Setup

The application was imported into the India data-centre account using the existing Zoho Creator CLI. The link name is hostel-management-system, the date format is dd-MMM-yyyy and the time zone is Asia/Kolkata. YAML and Deluge definitions are retained under apps/hostel-management. The live app now contains six form sections and a Dashboard page, added through the CLI page definition and verified on 20 October 2026.

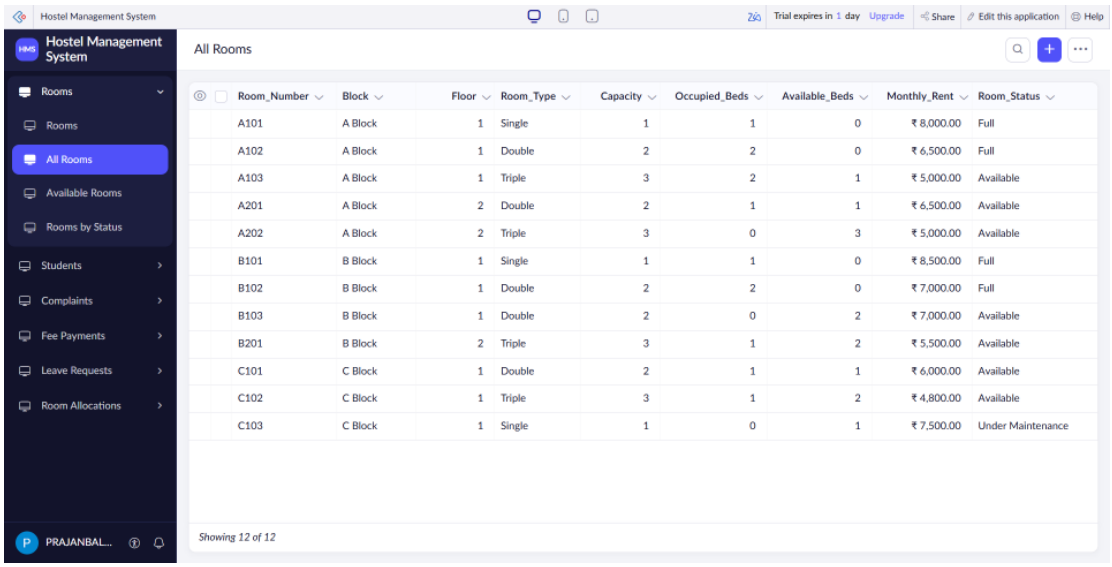


Figure 5.1: Live Application Home and Room Inventory

5.2 Forms and Data Capture

Rooms defines bed inventory and rent; Students records resident and guardian details; Room Allocations links each stay to a resident and room. Fee Payments contains a Fee Items subform for rent and other charges. Leave Requests records dates and decisions, while Complaints records the maintenance category, priority and resolution. Fourteen distinct source field types are used across the forms, including file upload, lookup and subform. The specification tables in Chapter 4 identify every field and its link name. Defaults and conditional rules are source-defined; screenshots alone are not treated as proof that every rule ran.

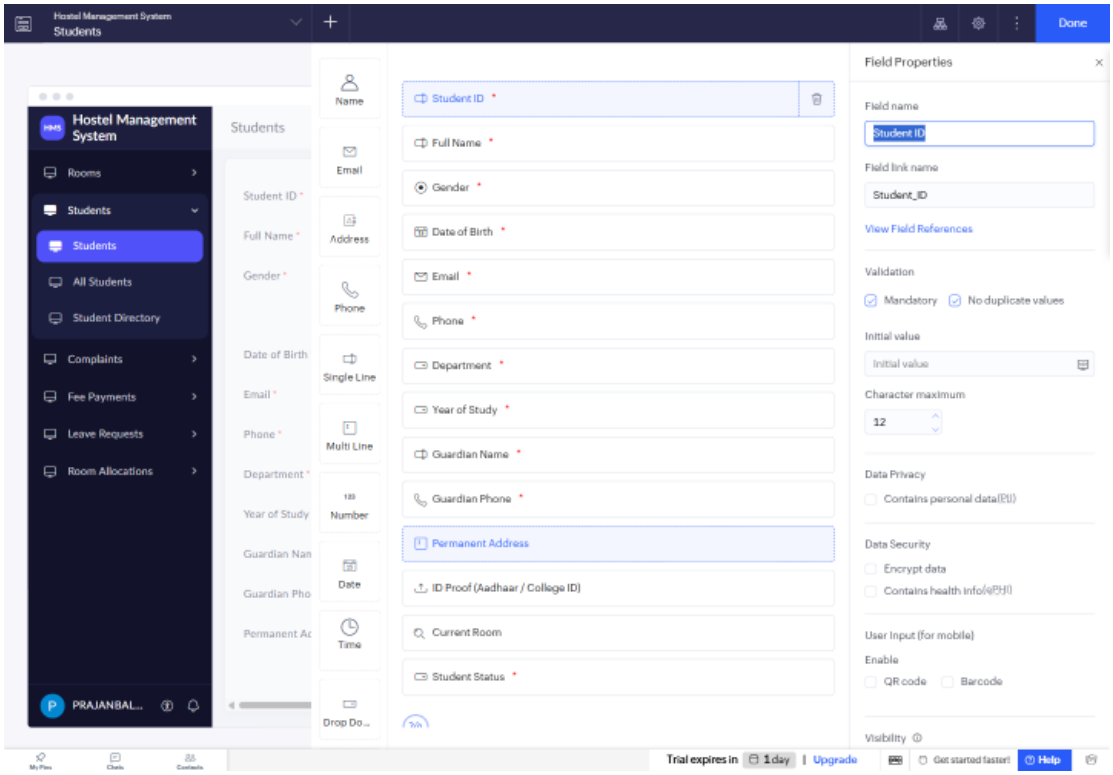


Figure 5.2: Students Form in the Creator Builder

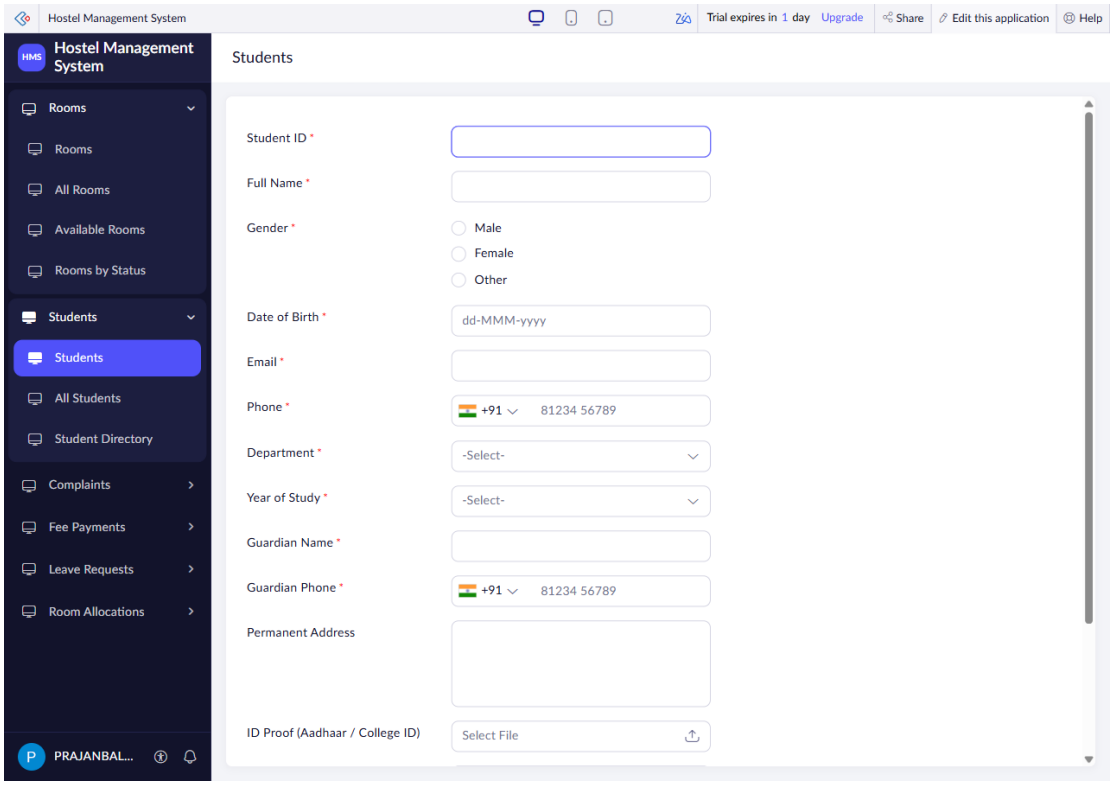


Figure 5.3: Students Form in Live Mode

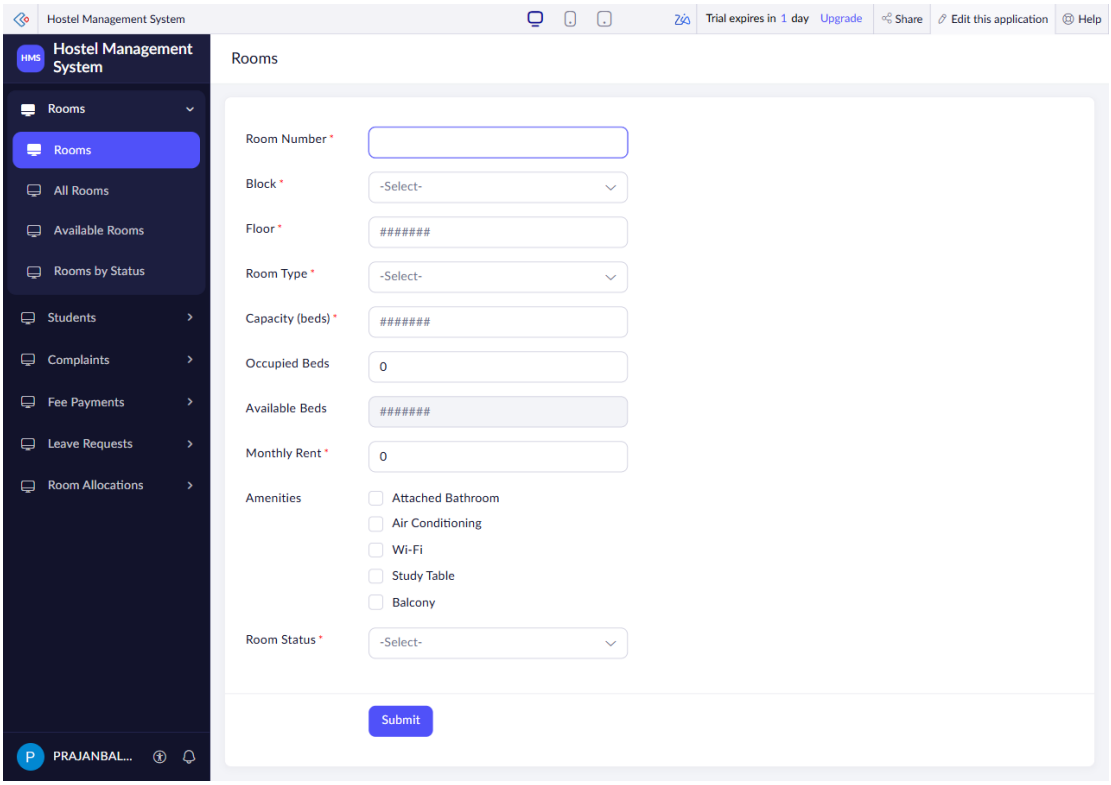


Figure 5.4: Rooms Form in Live Mode

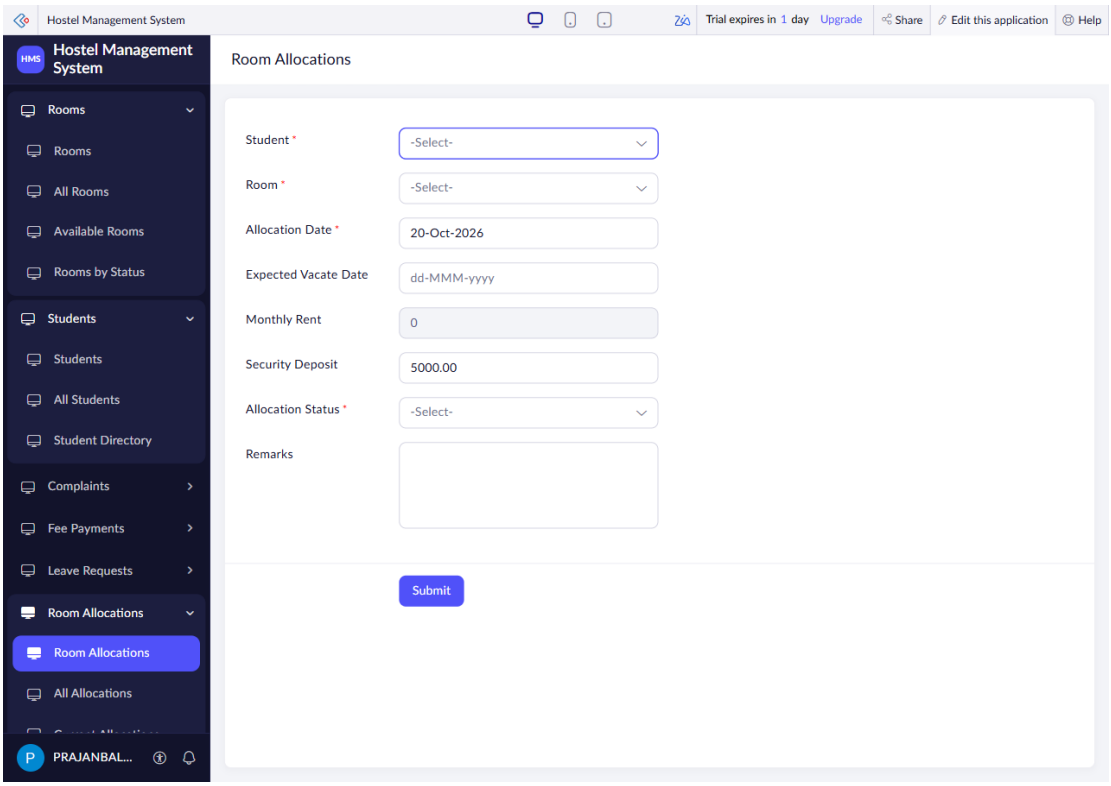


Figure 5.5: Room Allocations Form in Live Mode

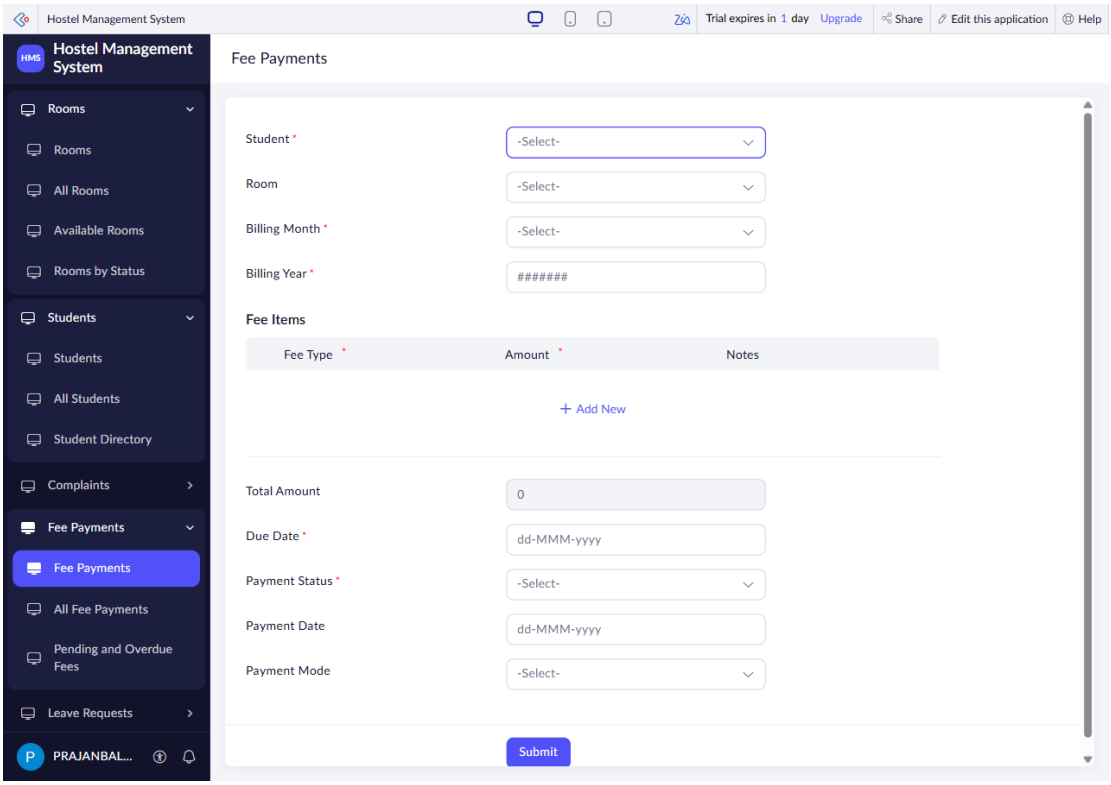


Figure 5.6: Fee Payments Form in Live Mode

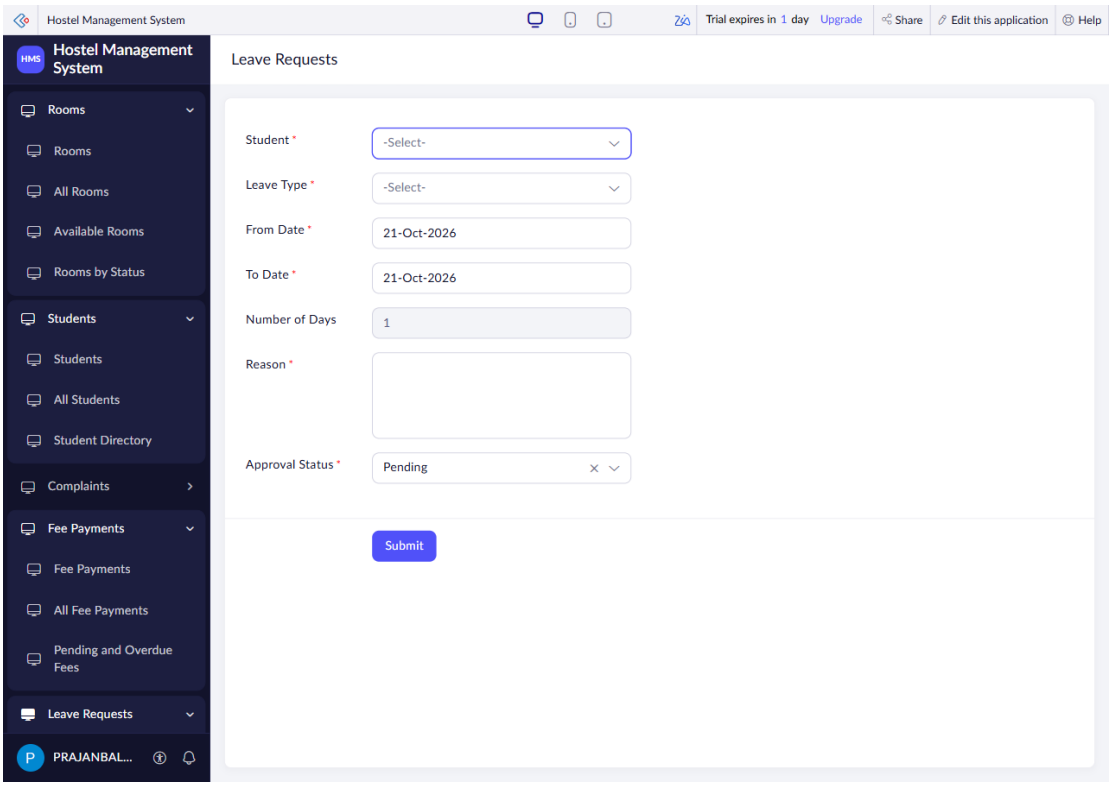


Figure 5.7: Leave Requests Form in Live Mode

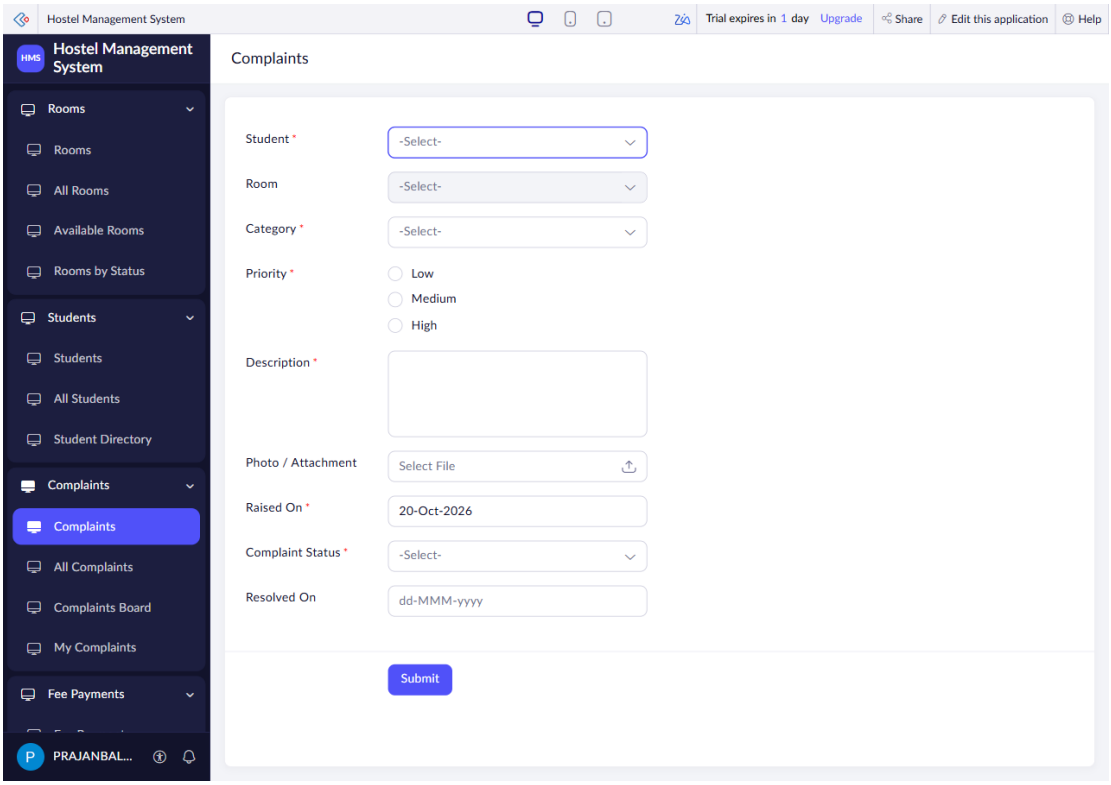


Figure 5.8: Complaints Form in Live Mode

5.3 Reports

Table 5.1: Reports Implemented

Report	Type	Source	Filter and sort	Actions	Profiles
All Allocations	List	Room_Allocations	All records; Allocation_Date desc	No custom action	Per source profile
All Complaints	List	Complaints	All records; Raised_On desc	No custom action	Per source profile
All Fee Payments	List	Fee_Payments	All records; Due_Date desc	No custom action	Per source profile
All Leave Requests	List	Leave_Requests	All records; From_Date desc	No custom action	Per source profile
All Rooms	List	Rooms	All records; Block asc, Room_Number asc	No custom action	Per source profile
All Students	List	Students	All records; Student_ID asc	No custom action	Per source profile
Available Rooms	List	Rooms	Room_Status == "Available"; Monthly_Rent asc	No custom action	Per source profile

Report	Type	Source	Filter and sort	Actions	Profiles
Complaints Board	Kanban	Complaints	All records; by Complaint_Status; Raised_On asc	No custom action	Per source profile
Current Allocations	List	Room_Allocations	Allocation_Status == "Active"; Room asc	No custom action	Per source profile
Leave Approvals	Kanban	Leave_Requests	All records; by Approval_Status; From_Date asc	No custom action	Per source profile
My Complaints	List	Complaints	Student.Email == zoho.loginuserid; Raised_On desc	No custom action	Per source profile
My Leave Requests	List	Leave_Requests	Student.Email == zoho.loginuserid; From_Date desc	No custom action	Per source profile
Pending and Overdue Fees	List	Fee_Payments	Payment_Status != "Paid"; Due_Date asc	No custom action	Per source profile
Rooms by Status	Kanban	Rooms	All records; by Room_Status	No custom action	Per source profile
Student Directory	Summary	Students	Student_Status == "Active"; Department asc, Full_Name asc	No custom action	Per source profile

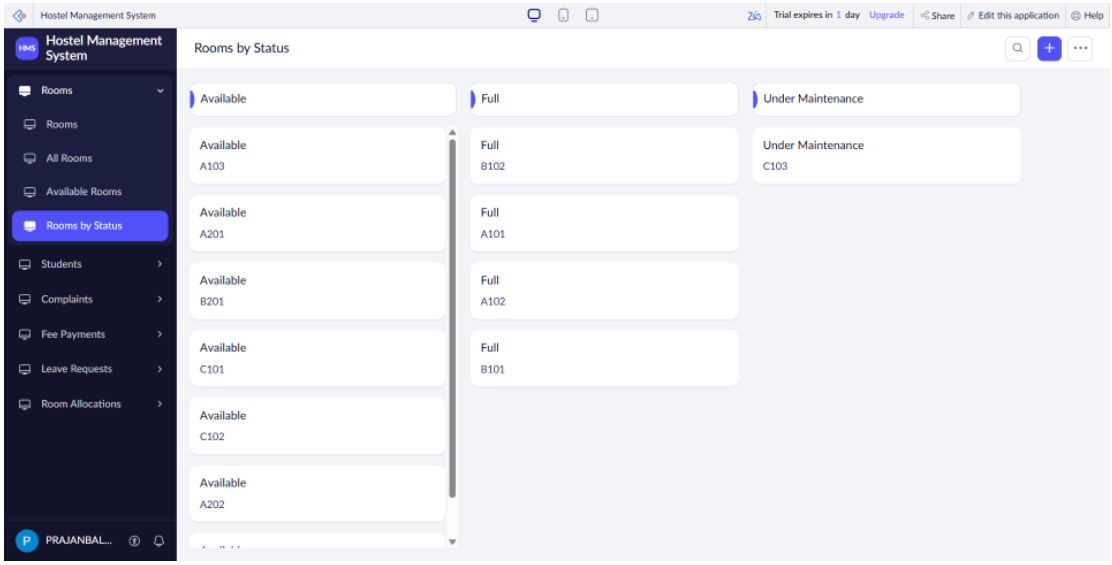


Figure 5.9: Rooms Grouped by Status in a Kanban Report

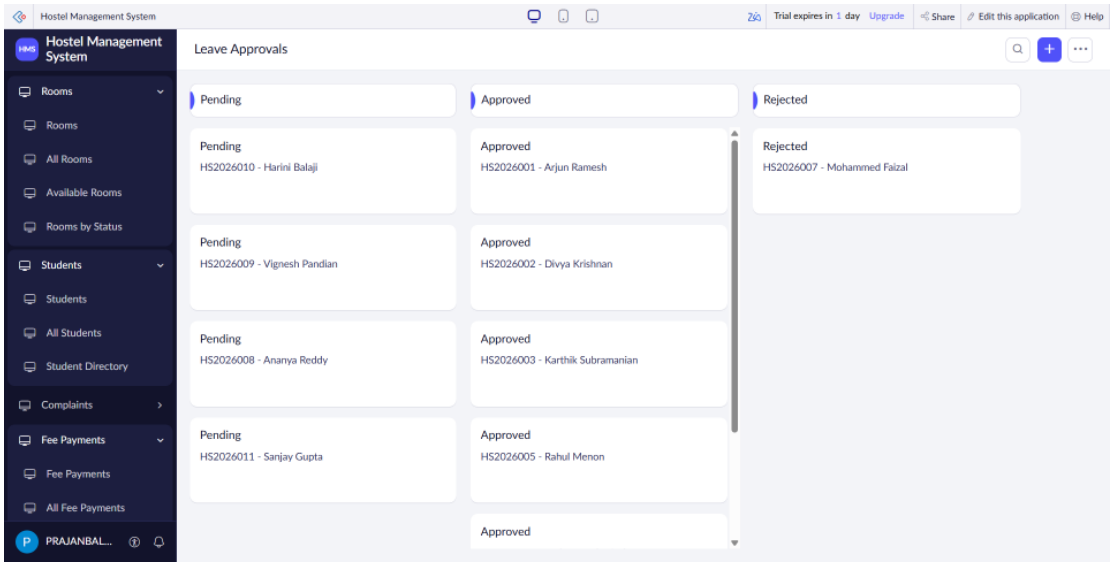


Figure 5.10: Leave Requests Grouped by Approval Status

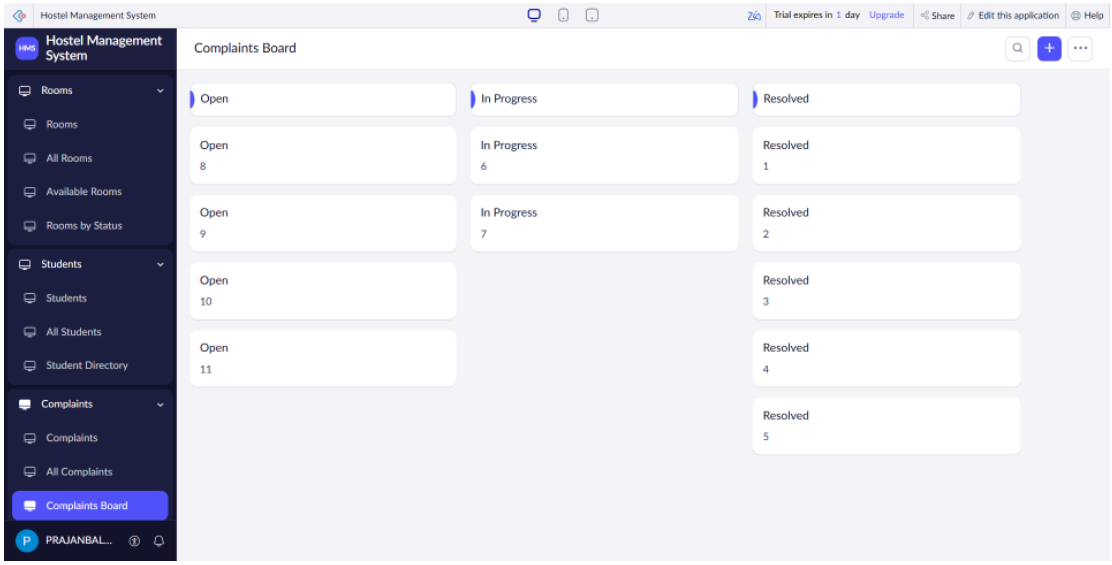


Figure 5.11: Complaints Grouped by Status

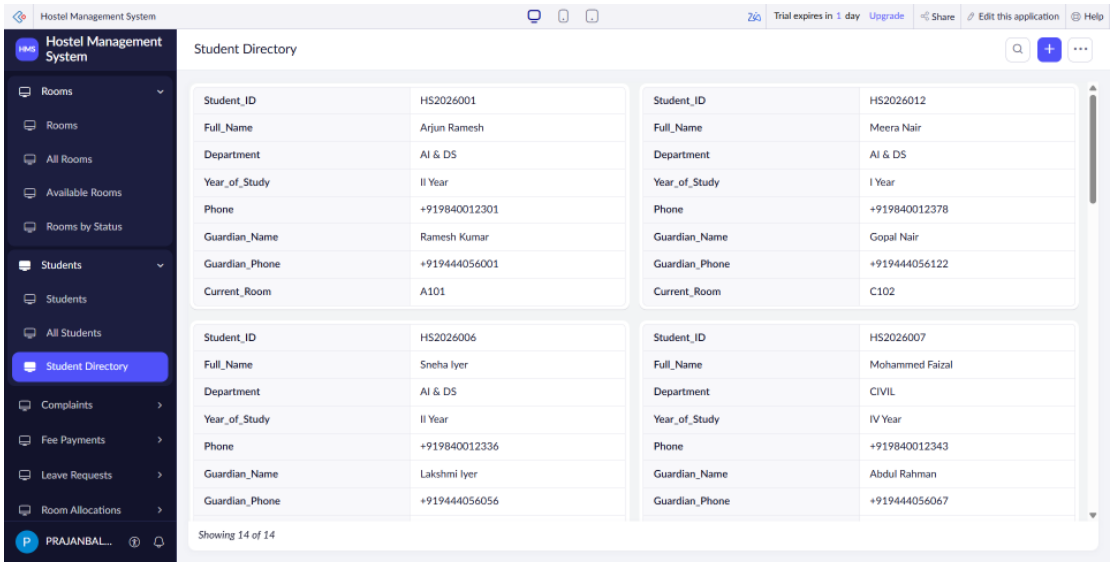


Figure 5.12: Active Student Directory Summary Report

5.4 Pages and Dashboards

The Dashboard page is implemented and was verified in the authenticated live app. Its six counters display 12 rooms, 12 free beds, 14 active residents, four pending leave requests, six unresolved complaints and two overdue bills. The free-bed calculation excludes rooms under maintenance. Four buttons open Room Allocations, Fee Payments, Leave Requests and Complaints. Two embedded reports show Leave Approvals and Pending and Overdue Fees. The page definition is retained in pages/Dashboard.yaml and compiled by the CLI into Creator page markup.

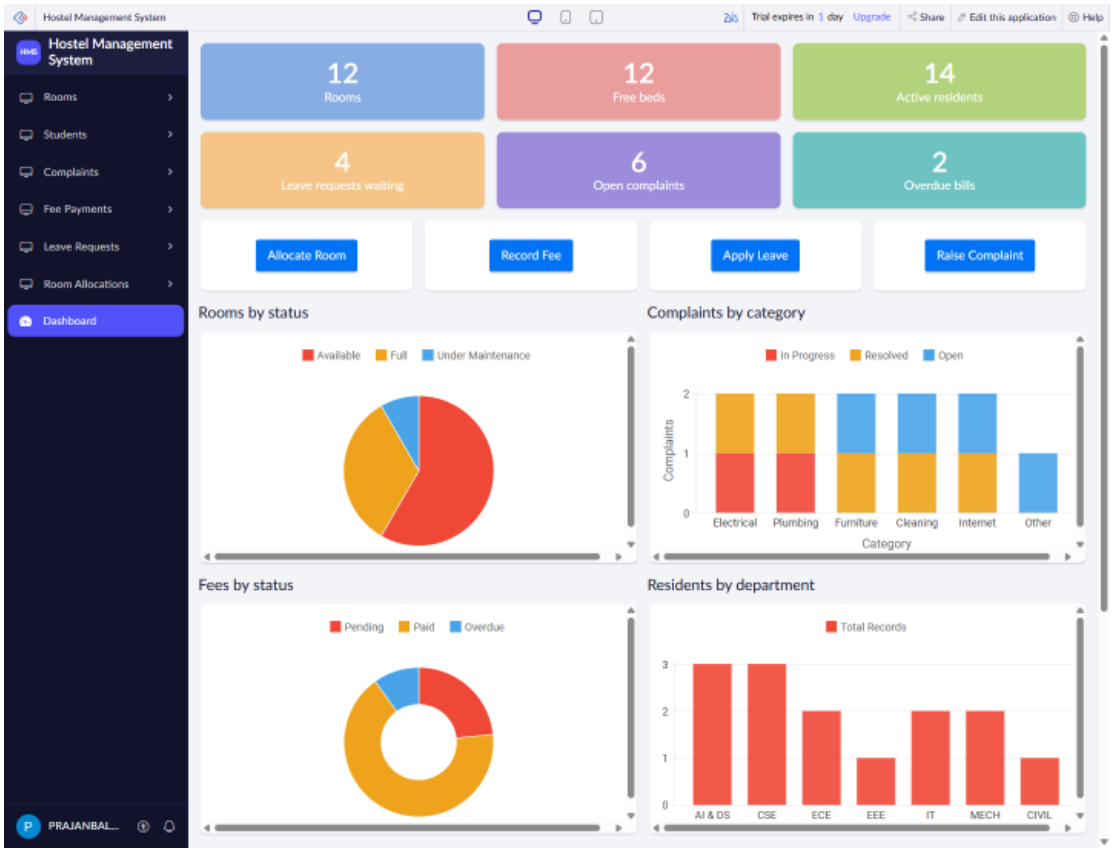


Figure 5.13: Live Dashboard Counters Quick Actions and Charts

5.5 Deluge Scripting

Table 5.2: Deluge Scripts Implemented

Script ID	Location	Purpose	Concepts
DS-09	Allocation validation	Prevent duplicate or full-room allocation	Criteria fetch, loops, alert, cancel
DS-02	Reusable function	Recount beds and set status	Count, assignment, if else
DS-21	Fee validation	Validate charges and total bill	Subform loop, sum, criteria
DS-26	Leave validation	Count days and detect overlap	Date difference, criteria
DS-01	Reusable function	Sum unpaid student fees	Criteria and sum
DS-32	Daily schedule	Mark past-due Pending bills and notify	Fetch, function call, sendmail

Code 5.1 Allocation validation excerpt

```
// Excerpt from Allocation_Validate
student = Students[ID == input.Student];
room = Rooms[ID == input.Room];
if(room.Room_Status == "Under Maintenance")
{
    alert "Room is under maintenance.";
    cancel submit;
}
// The full script additionally checks active-stay conflicts,
// capacity, dates and student status. See Appendix C.
```

Code 5.2 Occupancy refresh function

```
// Excerpt from refreshRoomOccupancy
room = Rooms[ID == roomId];
occupied = Room_Allocations[Room == roomId &&
    Allocation_Status == "Active"].count();
room.Occupied_Beds = occupied;
room.Available_Beds = room.Capacity - occupied;
if(room.Room_Status != "Under Maintenance")
{
    if(occupied >= room.Capacity)
    {
        room.Room_Status = "Full";
    }
    else { room.Room_Status = "Available"; }
}
// Complete function and guard appear in Appendix C.
```

The allocation validator fetches the related student and room, rejects conflicting active stays and checks capacity before submission. The occupancy function recounts active allocations rather than incrementing a stored counter, so repeated calls produce the same result [4]. All 12 room counts agreed with the active-allocation snapshot. This confirms consistency of the current records; a fresh add/edit/delete workflow test remains necessary.

5.6 Workflows and Automation

Table 5.3: Workflows Implemented

Workflow	Type	Trigger	Condition	Action	Script ID
Allocation_Added	Form	on success	add	Allocation - update room and notify student	DS-03
Allocation_Deleted	Form	on success	delete	Allocation - free the bed after delete	DS-04

Workflow	Type	Trigger	Condition	Action	Script ID
Allocation_Form_Load	Form	on load	add or edit	Allocation - set defaults	DS-05
Allocation_Room_Selected	Form	on user input	add or edit	Allocation - fetch room rent	DS-06
Allocation_Status_Changed	Form	on user input	add or edit	Allocation - show vacate date	DS-07
Allocation_Updated	Form	on success	edit	Allocation - update rooms after a change	DS-08
Allocation_Validate	Form	on validate	add or edit	Allocation - check room and student	DS-09
Complaint_Form_Load	Form	on load	add or edit	Complaint - set defaults	DS-10
Complaint_Raised	Form	on success	add	Complaint - alert warden on high priority	DS-11
Complaint_Resolved	Form	on validate	edit	Complaint - tell student it is resolved	DS-12
Complaint_Status_Changed	Form	on user input	add or edit	Complaint - show resolution fields	DS-13
Complaint_Student_Selected	Form	on user input	add or edit	Complaint - fetch room	DS-14
Complaint_Validate	Form	on validate	add or edit	Complaint - validate	DS-15
Fee_Form_Load	Form	on load	add or edit	Fee - set defaults	DS-16
Fee_Mode_Changed	Form	on user input	add or edit	Fee - show transaction reference	DS-17
Fee_Receipt	Form	on success	add or edit	Fee - email receipt	DS-18
Fee_Status_Changed	Form	on user input	add or edit	Fee - show payment details	DS-19
Fee_Student_Selected	Form	on user input	add or edit	Fee - fetch room and rent	DS-20

Workflow	Type	Trigger	Condition	Action	Script ID
Fee_Validate	Form	on validate	add or edit	Fee - validate and calculate total	DS-21
Leave_Dates_Changed	Form	on user input	add or edit	Leave - count days	DS-22
Leave_Decision	Form	on validate	edit	Leave - approval decision email	DS-23
Leave_Form_Load	Form	on load	add	Leave - new request defaults	DS-24
Leave_Review_Load	Form	on load	edit	Leave - warden review layout	DS-25
Leave_Validate	Form	on validate	add or edit	Leave - validate dates and overlap	DS-26
Room_Form_Load	Form	on load	add or edit	Room - set defaults	DS-27
Room_Type_Changed	Form	on user input	add or edit	Room - capacity from room type	DS-28
Room_Validate	Form	on validate	add or edit	Room - validate and update status	DS-29
Student_Form_Load	Form	on load	add or edit	Student - set defaults	DS-30
Student_Validate	Form	on validate	add or edit	Student - validate details	DS-31
Daily_Fee_Overdue_Check	Schedule	Daily 08:00 IST	Pending and past due	Daily fee overdue check	DS-32

Leave approval is implemented as a status field, warden review layout and decision email script. A native Creator approval process, Blueprint, batch workflow and report workflow are not implemented. The overdue schedule is configured to start on 21 October 2026 at 08:00 IST; its first scheduled execution had not occurred at inspection.

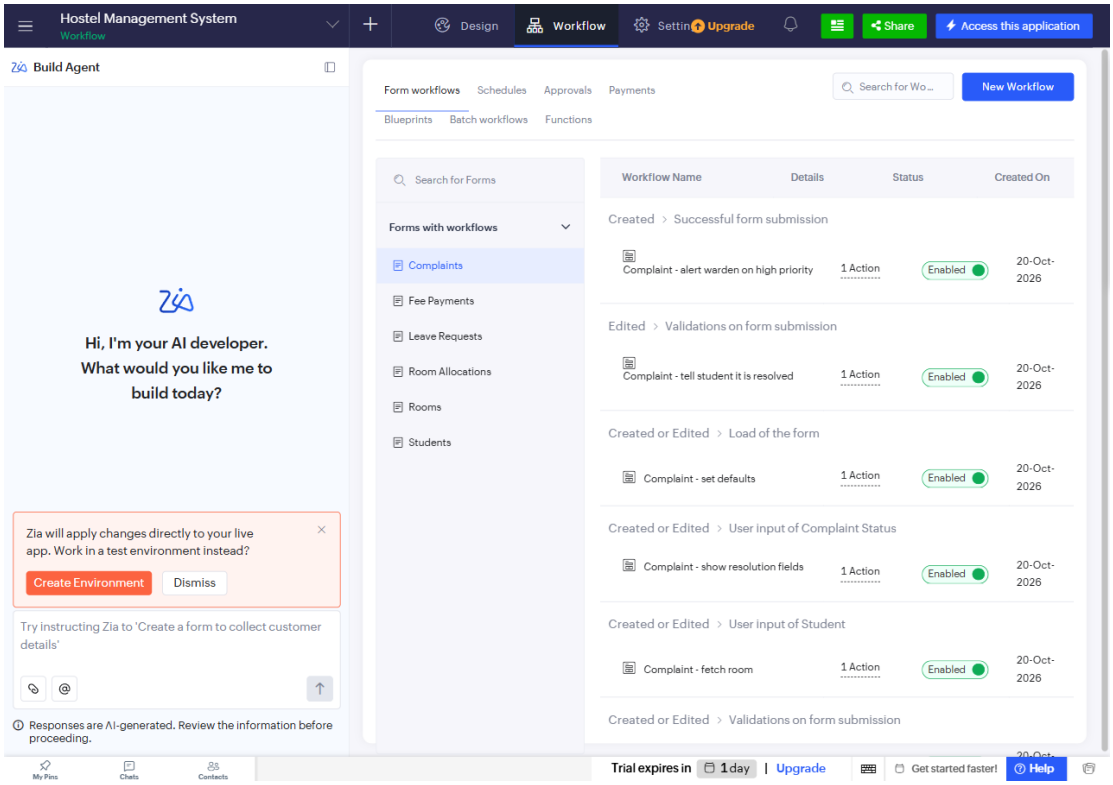


Figure 5.14: Form Workflow Definitions in the Creator Builder

5.7 Integrations

Table 5.4: Integrations Implemented

Integration	Service	Method	Authentication	Purpose and evidence
CLI data access	Creator REST API v2.1	GET metadata and report records	OAuth stored outside report	Counts and record checks completed
App deployment tooling	Creator import and builder	CLI and Playwright	Existing signed-in session	Live imported app and builder observed
Email notifications	Deluge sendmail	Workflow or schedule	Platform sender	Defined in source; delivery unverified
External business connector	None	Not implemented	Not applicable	No CRM, gateway or webhook claim

Code 5.3 Creator API read used to verify the live dataset

```
// Excerpt from the existing TypeScript verification client
const client = new CreatorClient(loadConfig());
const result = await client.getAllRecords(
    owner, app, "All_Rooms", { fieldConfig: "all" }
);
console.log(result.length); // 12 live room records
// OAuth credentials remain in the CLI configuration.
// No access token is included in this document.
```

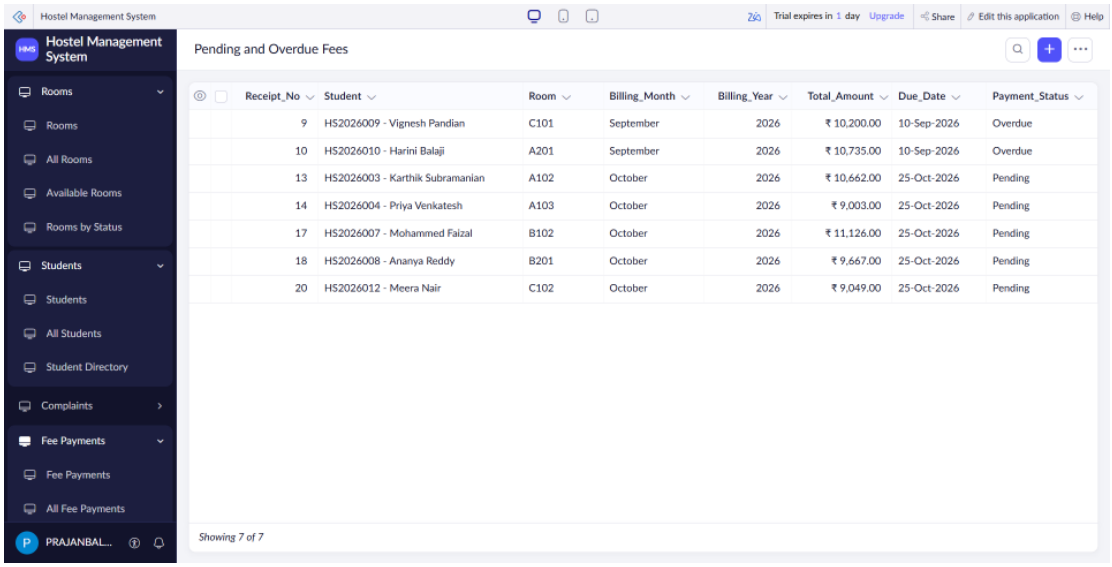


Figure 5.15: Live Pending and Overdue Fee Records Verified through the CLI

5.8 User Management and Role-Based Access

Three custom permission profiles separate Warden, Accountant and Student actions, in addition to Creator's Administrator. The Warden definition includes Dashboard access. The Accountant can export student data through a form-level permission while viewing the All Students report. The source role hierarchy places Warden and Accountant below CEO, and Student below Warden. The latest retained live export still lists the default CEO role; source role and test-user definitions must be synced and demonstrated before claiming role-isolation results. My Leave Requests and My Complaints use Student.Email == zoho.loginuserid [5].

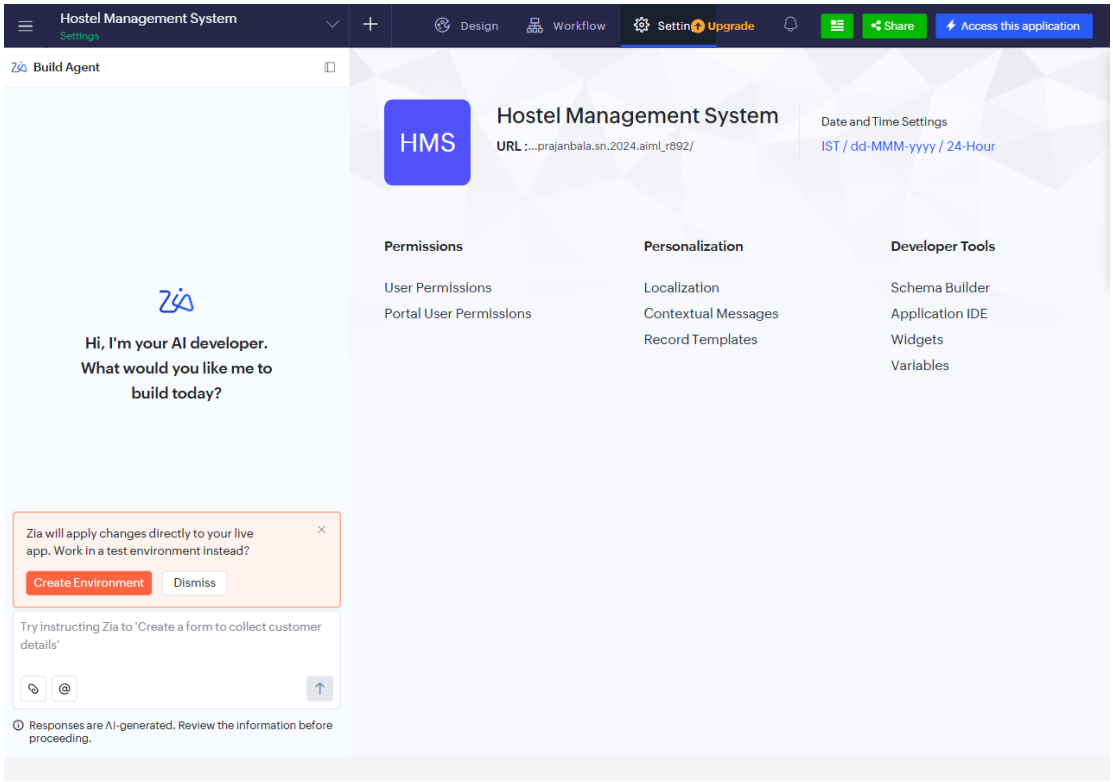


Figure 5.16: Application Settings with Permission Entry Points

5.9 AI and Analytics Features

Four charts are implemented on the live Dashboard: a pie chart of room status; complaints by category stacked by complaint status; a donut chart summing fee amounts by payment status; and a column chart of active residents by department. The fee chart uses amounts, rather than bill counts. These are native dashboard analytics; no separate pivot report or Zoho Analytics connector is claimed. No live Zia prediction, OCR or other AI field is used. External AI assistance in development and report preparation remains acknowledged.

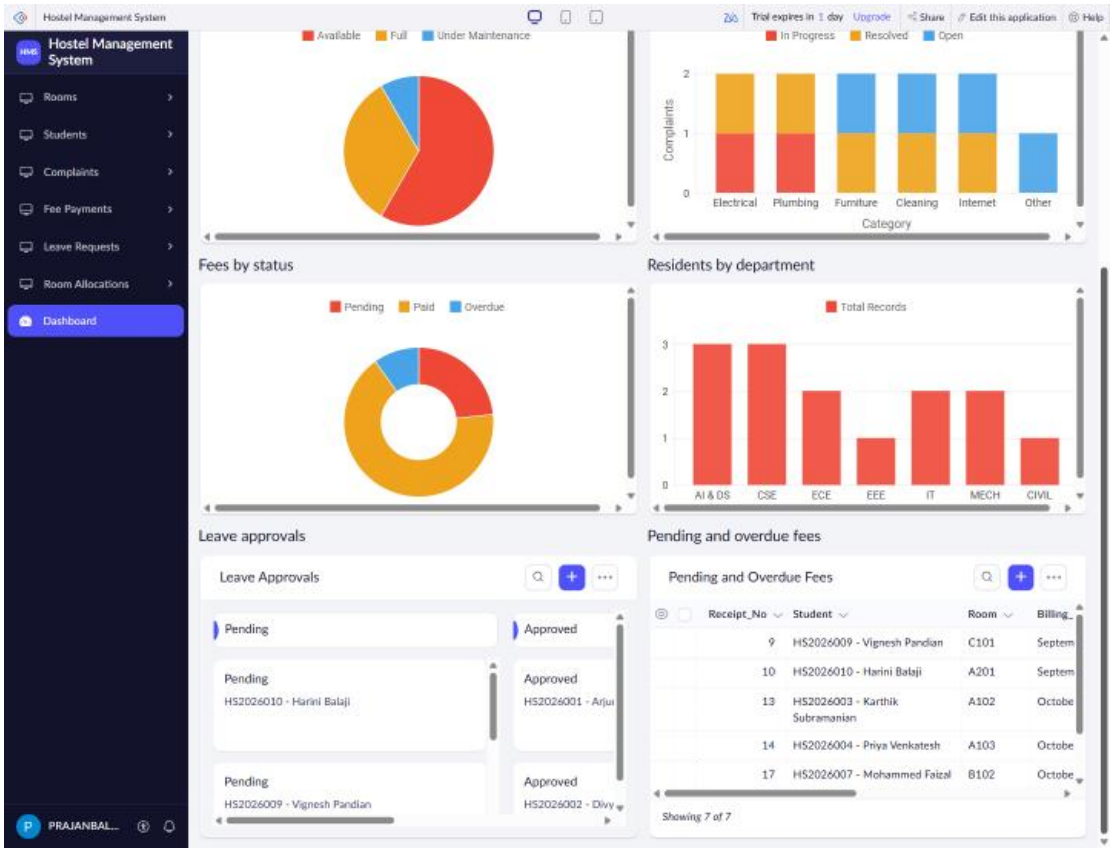


Figure 5.17: Dashboard Analytics and Embedded Operational Reports

5.10 Pro-Code Extension

The project's pro-code boundary is the TypeScript/Bun CLI that compiles YAML and Deluge definitions, imports the application, and reads metadata and records through Creator APIs. Browser operations use Playwright where needed. This tooling is outside the deployed student interface; no Catalyst service or embedded custom widget is deployed. Deluge functions remain part of Creator's low-code layer.

5.11 Mobile and Portal Access

Browser access was verified. The Zoho Creator native mobile app, offline behaviour and customer portal were not exercised, and no portal configuration is claimed. A desktop browser screenshot is not presented as evidence of native mobile support.
Mobile and portal evidence: not collected; browser access only was verified.

CHAPTER 6
TESTING AND DEBUGGING

6.1 Testing Strategy

Verification combined live metadata and record reads, Playwright screenshots, and source review. Twenty read-only inventory and consistency checks passed with the current count of one dashboard page. The dashboard acceptance check also passed: six counter values, four charts, four quick actions and two embedded reports were observed. Mutation tests, email delivery and separate-role login tests were not executed during report preparation, so their statuses remain distinct from the verified dashboard results.

6.2 Test Cases

Table 6.1: Test Cases

TC ID	FR	Test	Input or method	Expected	Actual	Status
TC-01	FR-01–06	Main forms count	Read live metadata or records	6	6	Pass
TC-02	FR-07	Report count	Read live metadata or records	15	15	Pass
TC-03	FR-10	Dashboard count	Read live metadata or records	1	1	Pass
TC-04	FR-02	All_Rooms count	Read live metadata or records	12	12	Pass
TC-05	FR-01	All_Students count	Read live metadata or records	15	15	Pass
TC-06	FR-03	All_Allocations count	Read live metadata or records	14	14	Pass
TC-07	FR-04	All_Fee_Payments count	Read live metadata or records	20	20	Pass
TC-08	FR-05	All_Leave_Requests count	Read live metadata or records	11	11	Pass

TC ID	FR	Test	Input or method	Expected	Actual	Status
TC-09	FR-06	All_Complaints count	Read live metadata or records	11	11	Pass
TC-10	FR-03	Room occupancy matches active allocation count	Read live metadata or records	True	True	Pass
TC-11	FR-02/03	Available beds equal capacity less occupancy	Read live metadata or records	True	True	Pass
TC-12	FR-03	No overcapacity room	Read live metadata or records	True	True	Pass
TC-13	FR-03	One active allocation per resident	Read live metadata or records	True	True	Pass
TC-14	FR-03	Current room agrees with active allocation	Read live metadata or records	True	True	Pass
TC-15	FR-03	No active allocation to maintenance room	Read live metadata or records	True	True	Pass
TC-16	FR-04	No duplicate student monthly fee bill	Read live metadata or records	True	True	Pass
TC-17	FR-05	Leave duration inclusive of start and end	Read live metadata or records	True	True	Pass
TC-18	FR-05	Rejected leave has warden remarks	Read live metadata or records	True	True	Pass
TC-19	FR-04	Paid fee rows have payment mode and date	Read live metadata or records	True	True	Pass
TC-20	FR-06	Resolved complaints have resolution date	Read live metadata or records	True	True	Pass
TC-21	FR-01	Invalid student birth date or duplicate phone	Submit invalid values	Clear rejection	Source reviewed only	Not run
TC-22	FR-03	Allocate full or maintenance room	Attempt invalid allocation	Submission blocked	Source reviewed only	Not run

TC ID	FR	Test	Input or method	Expected	Actual	Status
TC-23	FR-04	Mark UPI payment without reference	Submit missing reference	Submission blocked	Source reviewed only	Not run
TC-24	FR-05	Overlapping or reversed leave dates	Submit invalid range	Submission blocked	Source reviewed only	Not run
TC-25	FR-06	Resolve without notes or valid date	Submit incomplete resolution	Submission blocked	Source reviewed only	Not run
TC-26	FR-08	Notification and scheduled execution	Controlled workflow run	Expected email and status	No execution evidence	Not run
TC-27	FR-09	Profile and own-record isolation	Sign in separately as each profile	Only authorised records visible	Owner session only	Not run
TC-28	FR-10	Dashboard counters and components	Open Dashboard; compare record aggregates	6 counters; 4 charts; 4 actions; 2 reports	12 rooms; 12 beds; 14 residents; 4 leave; 6 complaints ; 2 overdue. Components visible.	Pass

6.3 Debugging Deluge Scripts

The project README and current source record import-time and round-trip corrections. Multi-field hide, show and disable commands were split into one statement per field because Creator retained only the last field. The fee-year check and leave-decision check use intermediate boolean variables to preserve logic when Creator removes grouping parentheses. Accountant export access was moved to a form-level permission. The dashboard and corrected component definitions appear in the retained live export. New execution logs and negative-submission runs were not collected.

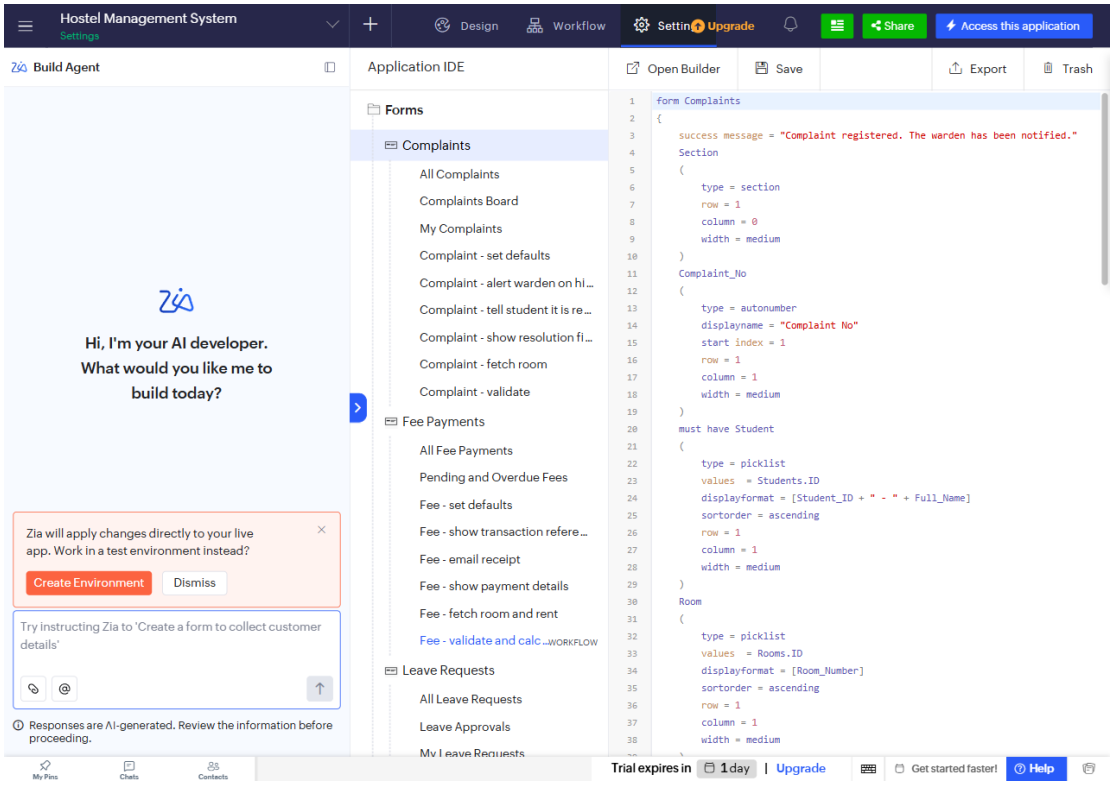


Figure 6.1: Imported Components Listed in the Application IDE

6.4 Defect Log

Table 6.2: Defect Log

Bug ID	Description	Cause	Recorded correction	Status
IMP-01	File upload import syntax rejected	Generated declaration syntax	Use Creator upload file syntax	Historical fix in README
IMP-02	decimal function return type rejected	Unsupported function type	pendingDues returns float	Historical fix in README
IMP-03	Kanban grouping rejected	Grouping field absent from columns	Include grouping status field	Historical fix in README
IMP-04	old values rejected in success scripts	Event scope mismatch	Move decision scripts to edit validation; recount rooms on update	Historical fix in README
GAP-01	Dashboard previously absent	Page not present in earlier snapshot	Dashboard, counters and four charts now live	Closed; browser verified

Bug ID	Description	Cause	Recorded correction	Status
GAP-02	Workflow and role tests incomplete	Read-only documentation session	Run controlled scenario and separate-role tests	Open
FIX-05	Multi-field UI rules lost fields	Creator retained only the final field	Use one hide, show or disable statement per field	Corrected in source and export
FIX-06	Boolean grouping changed on save	Creator dropped grouping parentheses	Use bad_year and valid_decision variables	Corrected in source and export
FIX-07	Accountant export permission misplaced	Export attached to report	Move Students export permission to form level	Corrected in source and export
FIX-08	Schedule initially inactive	Imported schedule needed saving	Schedule resaved through IDE; daily definition retained	Configuration corrected; first run pending

6.5 User Acceptance Testing

Table 6.3: User Acceptance Feedback

User	Role	Feedback	Action
Not collected	Warden or operational user	No genuine UAT response available	Schedule a guided resident-management demo
Not collected	Accountant or student	No genuine UAT response available	Record actual feedback after a separate-role demo

CHAPTER 7
DEPLOYMENT, SECURITY AND GOVERNANCE

7.1 Environments and Deployment Strategy

The application is present in the CLI production target and its authenticated live URL was verified. On 20 October 2026, the Dashboard page and charts were added and the corrected workflows and permission definition were reapplied. A retained export includes the Dashboard, schedule and updated scripts. Development and Stage environments were not demonstrated. Retain exports and source before further changes, and test a restored copy before treating that process as proven rollback.

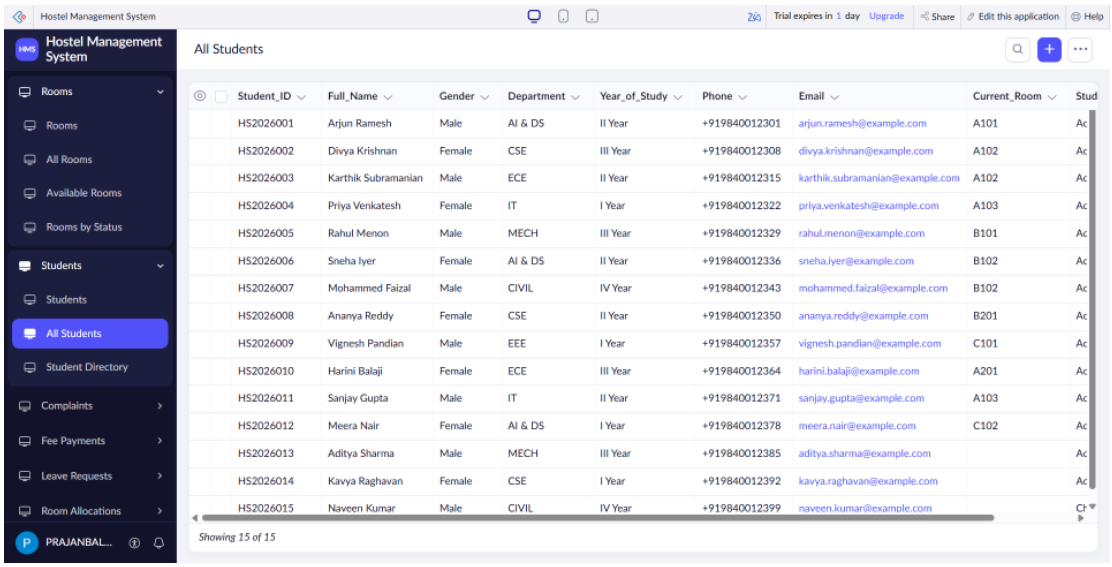


Figure 7.1: Populated Student Report in the Authenticated Live Application

Table 7.1: Deployment Checklist

No.	Deployment item	Status
1	All tests executed in Development or Stage	Not verified
2	Sample data removed or isolated	Demo dataset retained
3	Profiles and users tested in live app	Source profiles; role tests pending
4	Business-service connections authorised	No external business connector

No.	Deployment item	Status
5	Workflows and schedule executed	Definitions present; runtime tests pending
6	Authenticated live URL opened	Verified 20 October 2026
7	Deployment and source snapshot retained	CLI state and source retained

7.2 Performance Optimisation

Filtered reports reduce the records needed for operational tasks: Pending and Overdue Fees shows seven bills, while Available Rooms shows seven rooms. The occupancy function uses a criteria count. These are implementation choices, not measured performance improvements; no before-and-after benchmark was executed.

Table 7.2: Performance Optimisation Results

Technique	Component	Before	After	Measured result
Criteria count	Room occupancy function	No baseline recorded	Active allocations counted per room	12 of 12 room counts consistent; speed not measured
Filtered operational reports	Available Rooms and Pending Fees	All records view	7 rooms; 7 unpaid bills	Filtering verified; latency not benchmarked
Shared lookup values	Allocations and fee/complaint forms	Manual re-entry avoided by design	Related room and resident references	Current lookup consistency verified; effort not measured

7.3 Security Features

Table 7.3: Security Controls Implemented

Control	Applied or proposed location	Evidence and limit
OAuth credentials outside report	CLI configuration and client	Authenticated read completed; no token reproduced

Control	Applied or proposed location	Evidence and limit
Module permissions	Warden, Accountant, Student source profiles	Configuration reviewed; role login tests pending
Own-record criteria	My Leave Requests and My Complaints	Student.Email == zoho.loginuserid in source; runtime isolation untested
Input validation	Allocation, fee, student, leave and complaint scripts	Source reviewed; negative submission tests not run
Sensitive fields and uploads	Guardian details, ID Proof, attachments	Access review required; encrypted fields and audit settings not verified

7.4 Governance and Administration

The project retains YAML definitions, Deluge scripts, CLI state and application exports. The refreshed report includes the live Dashboard, four chart definitions and the latest source scripts. The current export was inspected for deployed components and the default role. User assignments, scheduled execution and audit-trail events should be verified during administration; the report does not substitute source definitions for a separate-role demonstration.

Audit evidence: a native audit-trail configuration and sample audit event were not verified.

CHAPTER 8
RESULTS AND DISCUSSION

8.1 Application Output

The following walkthrough follows existing records for HS2026012, Meera Nair. The student has an active allocation to C102, an October fee bill marked Pending, and a Cleaning complaint marked Open. These linked records demonstrate traceability across modules; they were already present and were not created as a new end-to-end transaction during this report session.

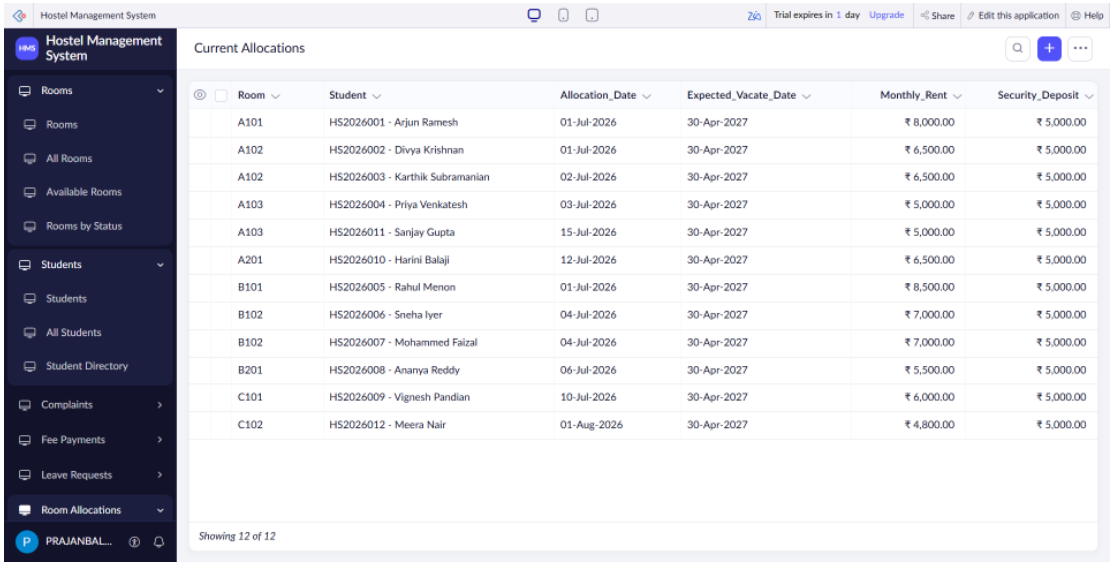


Figure 8.1: Existing Active Allocation for Meera Nair in C102

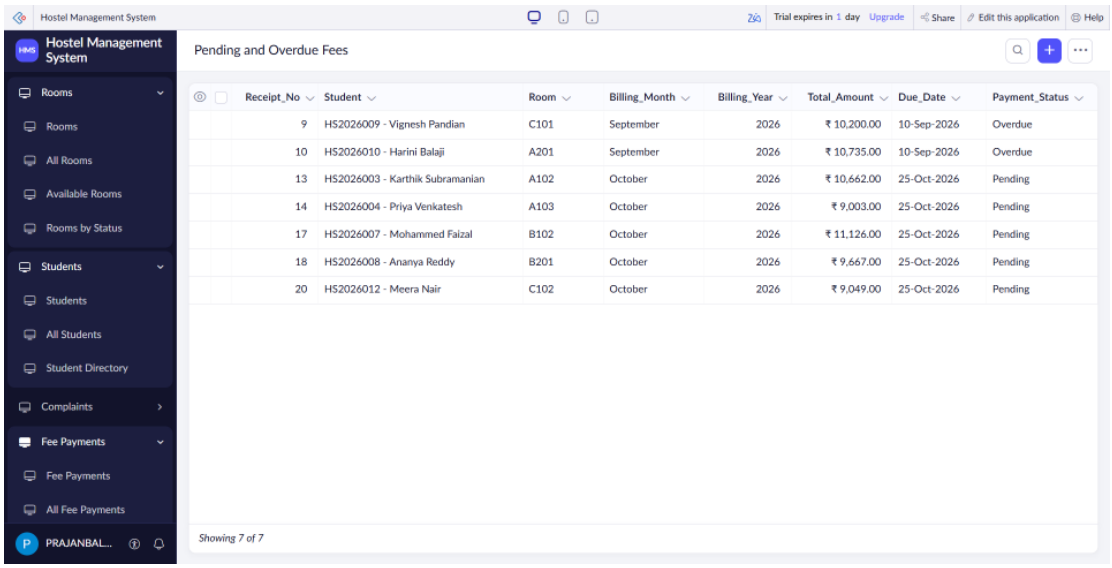


Figure 8.2: Existing October Fee for Meera Nair Marked Pending

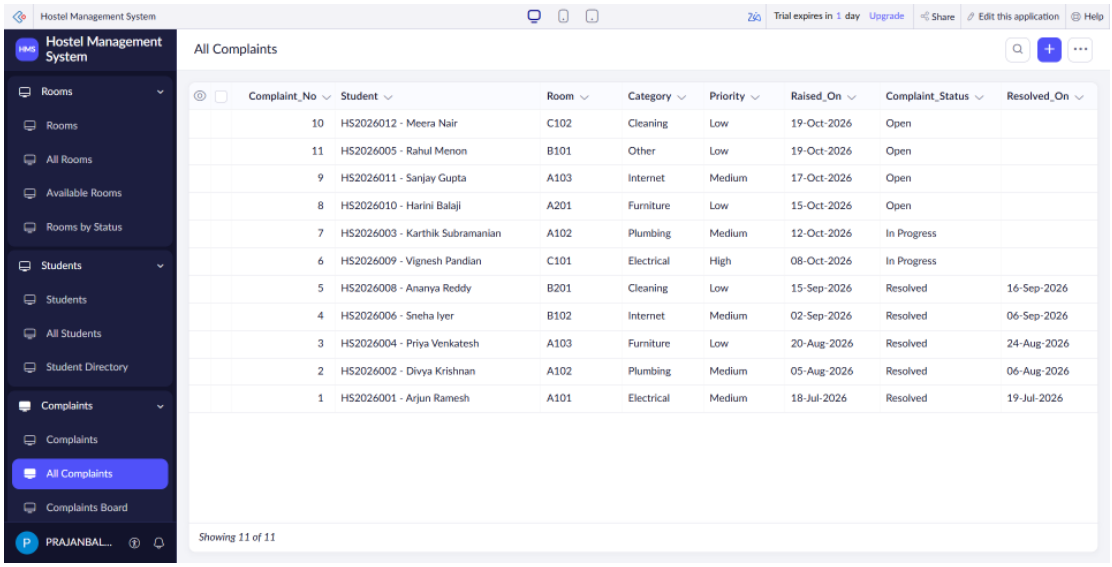


Figure 8.3: Existing Open Cleaning Complaint for Meera Nair in C102

8.2 Achievement of Objectives

Table 8.1: Mapping of Objectives to Outcomes

Objective	Implementation	Outcome or limit
Related data model	Six forms and Fee Items subform	Lookups and 83 main records observed
Allocation consistency	Validation and occupancy scripts	12 active stays match occupied-bed and current-room data
Operational reporting	15 reports plus Dashboard	Six counters, four charts and embedded pending-work reports verified
Automation	29 form workflows, two functions, one schedule	Source and builder present; full execution tests pending
Profile restrictions	Three custom source profiles and own-record reports	Separate-role demonstration outstanding

8.3 Comparison with the Existing System

Table 8.2: Existing System versus Proposed Application

Parameter	Illustrative manual process	Current application	Evidence
Room availability	Compare allocation register manually	All Rooms and Available Rooms	7 rooms offer 12 allocatable beds
Unpaid fees	Reconcile separate fee ledger	Pending and Overdue Fees	7 unpaid bills; INR 70442

Parameter	Illustrative manual process	Current application	Evidence
Pending work	Search messages or paper	Dashboard and operational boards	4 pending leave; 6 unresolved complaints
Data relationships	Re-enter names and room details	Lookup-backed records	20 read-only checks passed
Time and cost	No baseline measured	No benchmark measured	No percentage saving claimed

8.4 Limitations of the Low-Code Approach

The current account showed a trial expiry notice, so continued availability depends on its plan. Builder and API capabilities differ, requiring browser operations for some configuration. The Dashboard now provides the required overview, while an owner session alone cannot prove profile isolation. Allocation updates recount all rooms and should be measured before scaling. Platform-specific definitions create a migration cost. No quota-exhaustion or production-scale performance claim is made.

8.5 Reflection on the No-Code → Low-Code → Pro-Code Spectrum

Field definitions, lookup configuration and report layouts lie toward the no-code end. Deluge validation, calculations, record fetches and notifications form the low-code portion. The external TypeScript CLI and Playwright browser driver form the pro-code portion. A future move to a custom service would be justified by measured complexity, integration requirements or processing needs, not by the presence of scripts alone.

CHAPTER 9
CONCLUSION AND FUTURE ENHANCEMENTS

9.1 Conclusion

The Hostel Management System is deployed with six related forms, fifteen reports, a Dashboard page and realistic sample data. The dashboard's counters, four charts, quick actions and embedded reports were verified in the live app. Twenty read-only inventory and consistency checks passed, and the dashboard acceptance check passed. The report also includes the corrected source scripts. Controlled workflow execution, role-isolation tests and genuine user feedback remain to be documented for the assessed demonstration.

9.2 Future Enhancements

- Extend the existing Dashboard with billing-period filters, drill-downs and historical occupancy trends.
- Run controlled end-to-end and negative tests with separate Warden, Accountant and Student users, then record two genuine UAT responses.
- Evaluate payment-gateway integration and payment reconciliation after workflow execution and access testing are complete.

9.3 Learning Outcomes and Career Relevance

The project develops skills in schema design, workflow modelling, Deluge scripting, API access and evidence-based testing. It also demonstrates the responsibility to review AI-generated rules and to distinguish configured features from observed execution. These skills apply to low-code development, automation engineering and solutions consulting. Future work should strengthen testing, governance and user validation before expanding the feature set.

REFERENCES

[1] A. Bucaioni, A. Cicchetti and F. Ciccozzi, "Modelling in low-code development: a multi-vocal systematic review," Software and Systems Modeling, vol. 21, pp. 1959–1981, 2022, doi: 10.1007/s10270-021-00964-0.

[2] Zoho Corporation, "Creating and managing form workflows," Zoho Creator Help. Available: https://help.zoho.com/portal/en/kb/creator/developer-guide/workflows/create-and-manage-form-workflows/articles/create-new-form-workflow (accessed 20 October 2026).

[3] Zoho Corporation, "Scripting for Subforms," Zoho Creator. Available: https://www.zoho.com/creator/videos/scripting-for-subforms.html (accessed 20 October 2026).

[4] Zoho Corporation, "For each record," Deluge Help. Available: https://www.zoho.com/deluge/help/data-access/for-each-record.html (accessed 20 October 2026).

[5] Zoho Corporation, "Understanding user permissions," Zoho Creator Help. Available: https://help.zoho.com/portal/en/kb/creator/developer-guide/application-settings/manage-and-configure-user-permissions/articles/understand-user-permissions (accessed 20 October 2026).

[6] Zoho Corporation, "Get Records API v2.1," Zoho Creator Help. Available: https://www.zoho.com/creator/help/api/v2.1/get-records.html (accessed 20 October 2026).

[7] Microsoft, "BrowserType connectOverCDP and Screenshots," Playwright documentation. Available: https://github.com/microsoft/playwright/blob/main/docs/src/screenshots.md (accessed 20 October 2026).

APPENDIX A – ZOHO CREATOR FEATURE COVERAGE CHECKLIST

Table A.1: Feature Coverage Checklist

Unit	Zoho Creator Feature	Done	Where in the Application	Report Section	Figure / Code No.
I	Application created & sections/menu organised	Y	Six form sections and Dashboard	5.1, 5.4	Figs 5.1, 5.13
II	Forms with ≥ 8 distinct field types	Y	14 source field types	5.2	Tables 4.2–4.8
II	Lookup field (relationship between forms)	Y	Studentand Room lookups	4.2, 5.2	Fig 4.2
II	Subform (parent–child data)	Y	Fee Items	4.3, 5.2	Table 4.6
II	Field validations / layout rules	Partial	Validation and layout scripts; negative tests pending	5.2, 6.2	Appendix C
II	List report with filter, grouping, custom action	Partial	Filtered lists; no custom action	5.3	Table 5.1
II	Kanban / Calendar / Timeline / Map report	Y	Rooms, leave and complaint boards	5.3	Figs 5.9–5.11
II	Pivot table / chart report	Partial	Four dashboard charts; no separate pivot report	5.9	Fig 5.17
II	Page / dashboard built with UI Builder	Y	Dashboard with counters actions and embedded reports	5.4	Fig 5.13
II	Deluge – form events (validate / success / field actions)	Partial	Form event scripts present; execution tests pending	5.5	Appendix C
II	Deluge – functions, fetch, loops, collections	Y	Two functions and source loops	5.5	Code 5.2
II	Email / in-app notifications	Partial	sendmail scripts; delivery not verified	5.6	Appendix C
II	Form workflow	Y	29 form workflows	5.6	Table 5.3
II	Scheduled workflow	Partial	Daily 08:00 schedule; starts 21 Oct	5.6	Appendix C

Unit	Zoho Creator Feature	Done	Where in the Application	Report Section	Figure / Code No.
II	Approval process	Partial	Scripted leave decision; no native approval	5.6	Fig 5.10
II	Blueprint	N	No Blueprint	5.6	Not implemented
II	Batch / report workflow	N	No batch or report workflow	5.6	Not implemented
II	Testing & debugging evidence	Partial	20 live checks; runtime negatives pending	6	Table 6.1
III	Integration with a Zoho service	N	No separate Zoho business-service connector	5.7	Table 5.4
III	Connection / REST API call (invokeurl)	N	No Deluge invokeurl connection	5.7	Table 5.4
III	Webhook or Creator REST API	Y	Creator API used by CLI	5.7	Code 5.3
III	Users, roles and permission profiles	Partial	Three permission profiles; role and test-user source definitions	5.8	Table 4.9
III	Record-level access criteria / customer portal	Partial	Own-record criteria; isolation untested	5.8	Source reports
III	Zia / AI feature	N	No live Zia feature	5.9	Not implemented
III	Analytics (charts / Zoho Analytics)	Y	Four native charts on Dashboard	5.9	Fig 5.17
IV	Performance optimisation applied & measured	N	No before-and-after benchmark	7.2	Table 7.2
IV	Deployment across environments	N	Production only; no environment promotion	7.1	Table 7.1
IV	Security controls (encryption, OAuth, permissions)	Partial	OAuth and source permissions	7.3	Table 7.3

Unit	Zoho Creator Feature	Done	Where in the Application	Report Section	Figure / Code No.
IV	Governance (audit trail, administration)	Partial	Source and export history; audit unverified	7.4	Section 7.4
V	Pro-code extension (Catalyst / widgets / custom functions)	Y	External TypeScript CLI and Creator API	5.10	Code 5.3
V	Low-code limitations & spectrum analysis	Y	Limits and spectrum discussed	8.4, 8.5	Sections 8.4–8.5
—	Mobile app / portal access	N	No mobile or portal verification	5.11	Not verified

Coverage summary: 12 evidenced items, 12 partial items and 8 absent or unverified items out of 32. Partial items are not counted as complete.

APPENDIX B – COURSE OUTCOME MAPPING

Table B.1: Mapping of Course Outcomes to Report Chapters

CO	Course Outcome Statement	Evidenced In	Self-Assessed Level	Examiner Level
CO1	Explain the principles, architecture, and business value of low-code application development using Zoho Creator.	Ch. 1, 2	2	
CO2	Develop business applications using Zoho Creator forms, reports, workflows, and Deluge scripting.	Ch. 4, 5 (5.1–5.6), 6	2	
CO3	Integrate Zoho Creator applications with Zoho services and external systems using APIs, connections, and role-based security.	Ch. 4.6, 5.7–5.9	1	
CO4	Deploy, secure, and manage Zoho Creator applications across development and production environments following governance best practices.	Ch. 7	1	
CO5	Evaluate low-code development strategies, AI-assisted application development, and the transition from citizen developer to professional developer.	Ch. 5.9, 5.10, 8.4, 8.5, 9.3	2	

APPENDIX C – COMPLETE DELUGE SOURCE CODE

C.1 Complete Deluge Source from the Project

The following 32 scripts are reproduced from the local project definitions used for deployment. The live IDE lists the same component names. This appendix is a source snapshot, not a byte-for-byte export of every remote script. Header comments identify form events and schedule timing; they are metadata consumed by the CLI.

C.2 DS-01 pendingDues

```
float pendingDues(int studentId)
{
    // Total of every fee record for the student that is not yet
paid (Pending or Overdue).
    return ifnull(Fee_Payments[Student == studentId &&
Payment_Status != "Paid"].sum(Total_Amount),0.0);
}
```

C.3 DS-02 refreshRoomOccupancy

```
void refreshRoomOccupancy(int roomId)
{
    // Recounts a room's active allocations and keeps its bed
counts and status in step.
    // Called after an allocation is added, edited or deleted.
    room = Rooms[ID == roomId];
    if(room.count() > 0)
    {
        occupied = Room_Allocations[Room == roomId &&
Allocation_Status == "Active"].count();
        room.Occupied_Beds = occupied;
        room.Available_Beds = room.Capacity - occupied;
        // A room under maintenance stays that way until the
warden reopens it.
        if(room.Room_Status != "Under Maintenance")
        {
            if(occupied >= room.Capacity)
            {
                room.Room_Status = "Full";
            }
            else
            {
                room.Room_Status = "Available";
            }
        }
    }
}
```

C.4 DS-03 Allocation_Added

```
// form: Room_Allocations
// event: on_success
// record_event: add
// label: Allocation - update room and notify student
// ---
```

```
// Status update: recount the room's beds and record the student's
room.
thisapp.refreshRoomOccupancy(input.Room);
if(input.Allocation_Status == "Active")
{
    student = Students[ID == input.Student];
    room = Rooms[ID == input.Room];
    student.Current_Room = input.Room;
    // Notification
    sendmail
    [
        from :zoho.adminuserid
        to :student.Email
        subject :"Hostel room " + room.Room_Number + " allocated"
        message :"Dear " + student.Full_Name + ",<br><br>You have
been allocated room <b>" + room.Room_Number + "</b> (" +
room.Block + ", " + room.Room_Type + ") from " +
input.Allocation_Date + ".<br>Monthly rent: Rs. " +
input.Monthly_Rent + "<br>Security deposit: Rs. " +
input.Security_Deposit + "<br><br>Regards,<br>Hostel Office"
    ]
}
```

C.5 DS-04 Allocation_Deleted

```
// form: Room_Allocations
// event: on_success
// record_event: delete
// label: Allocation - free the bed after delete
// ---
thisapp.refreshRoomOccupancy(input.Room);
student = Students[ID == input.Student];
if(student.Current_Room == input.Room)
{
    student.Current_Room = null;
}
```

C.6 DS-05 Allocation_Form_Load

```
// form: Room_Allocations
// event: on_load
// record_event: add_or_edit
// label: Allocation - set defaults
// ---
if(input.Allocation_Date == null)
{
    input.Allocation_Date = zoho.currentdate;
}
if(input.Allocation_Status == null)
{
    input.Allocation_Status = "Active";
}
if(input.Security_Deposit == null || input.Security_Deposit == 0)
{
    input.Security_Deposit = 5000;
}
// Rent always comes from the room.
disable Monthly_Rent;
```

```
if(input.Allocation_Status != "Vacated")
{
    hide Vacated_On;
}
```

C.7 DS-06 Allocation_Room_Selected

```
// form: Room_Allocations
// event: on_user_input
// field: Room
// record_event: add_or_edit
// label: Allocation - fetch room rent
// ---
// Fetches related information from the chosen room.
if(input.Room != null)
{
    room = Rooms[ID == input.Room];
    input.Monthly_Rent = room.Monthly_Rent;
    if(room.Room_Status == "Under Maintenance")
    {
        alert "Room " + room.Room_Number + " is under maintenance.
Choose another room.";
    }
    else if(ifnull(room.Available_Beds,room.Capacity) <= 0)
    {
        alert "Room " + room.Room_Number + " is full. Choose
another room.";
    }
}
else
{
    input.Monthly_Rent = null;
}
```

C.8 DS-07 Allocation_Status_Changed

```
// form: Room_Allocations
// event: on_user_input
// field: Allocation_Status
// record_event: add_or_edit
// label: Allocation - show vacate date
// ---
// Conditional behaviour: the vacate date only applies to a
vacated allocation.
if(input.Allocation_Status == "Vacated")
{
    show Vacated_On;
    if(input.Vacated_On == null)
    {
        input.Vacated_On = zoho.currentdate;
    }
}
else
{
    input.Vacated_On = null;
    hide Vacated_On;
}
```

C.9 DS-08 Allocation_Updated

```
// form: Room_Allocations
// event: on_success
// record_event: edit
// label: Allocation - update rooms after a change
// ---
// The previous room isn't known after saving (Creator allows
old.* only while validating),
// so recount every room: that covers a student who moved from
another room.
for each r in Rooms[ID != 0]
{
    thisapp.refreshRoomOccupancy(r.ID);
}
student = Students[ID == input.Student];
if(input.Allocation_Status == "Active")
{
    student.Current_Room = input.Room;
}
else if(Room_Allocations[Student == input.Student &&
Allocation_Status == "Active"].count() == 0)
{
    student.Current_Room = null;
}
```

C.10 DS-09 Allocation_Validate

```
// form: Room_Allocations
// event: on_validate
// record_event: add_or_edit
// label: Allocation - check room and student
// ---
error = "";
student = Students[ID == input.Student];
room = Rooms[ID == input.Room];
if(input.Expected_Vacate_Date != null &&
input.Expected_Vacate_Date <= input.Allocation_Date)
{
    error = "Expected vacate date must be after the allocation
date.";
}
else if(input.Allocation_Status == "Vacated" && input.Vacated_On
== null)
{
    error = "Enter the date the student vacated the room.";
}
else if(input.Vacated_On != null && input.Vacated_On <
input.Allocation_Date)
{
    error = "Vacated date cannot be before the allocation date.";
}
else if(input.Allocation_Status == "Active")
{
    if(student.Student_Status != "Active")
    {
        error = student.Full_Name + " is checked out. Set the
student back to Active first.";
    }
```

```
    else if(room.Room_Status == "Under Maintenance")
    {
        error = "Room " + room.Room_Number + " is under
maintenance.";
    }
    else
    {
        // Duplicate check: a student holds only one active
allocation.
        for each a in Room_Allocations[Student == input.Student &&
Allocation_Status == "Active"]
        {
            if(a.ID != input.ID)
            {
                error = student.Full_Name + " already has an
active room allocation.";
            }
        }
        // Capacity check: count the other active allocations in
this room.
        others = 0;
        for each a in Room_Allocations[Room == input.Room &&
Allocation_Status == "Active"]
        {
            if(a.ID != input.ID)
            {
                others = others + 1;
            }
        }
        if(error == "" && others >= room.Capacity)
        {
            error = "Room " + room.Room_Number + " is full (" +
room.Capacity + " beds).";
        }
    }
}
if(error != "")
{
    alert error;
    cancel submit;
}
// The rent is copied from the room so it is never typed twice.
input.Monthly_Rent = room.Monthly_Rent;
```

C.11 DS-10 Complaint_Form_Load

```
// form: Complaints
// event: on_load
// record_event: add_or_edit
// label: Complaint - set defaults
// ---
if(input.Raised_On == null)
{
    input.Raised_On = zoho.currentdate;
}
if(input.Complaint_Status == null)
{
    input.Complaint_Status = "Open";
}
```

```
if(input.Priority == null)
{
    input.Priority = "Medium";
}
disable Room;
if(input.Complaint_Status != "Resolved")
{
    hide Resolved_On;
    hide Resolution_Notes;
}
```

C.12 DS-11 Complaint_Raised

```
// form: Complaints
// event: on_success
// record_event: add
// label: Complaint - alert warden on high priority
// ---
// Notification: high-priority complaints go straight to the
warden.
if(input.Priority == "High")
{
    student = Students[ID == input.Student];
    room = Rooms[ID == input.Room];
    sendmail
    [
        from :zoho.adminuserid
        to :zoho.adminuserid
        subject :"High priority " + input.Category + " complaint -
Room " + ifnull(room.Room_Number,"-")
        message :student.Full_Name + " (" + student.Student_ID +
") reported:<br><br>" + input.Description
    ]
}
```

C.13 DS-12 Complaint_Resolved

```
// form: Complaints
// event: on_validate
// record_event: edit
// label: Complaint - tell student it is resolved
// ---
// Runs while validating the edit, where old.* is available; only
sends once the
// resolution details are complete and valid.
if(input.Complaint_Status == "Resolved" && old.Complaint_Status !=
"Resolved" && input.Resolved_On != null &&
ifnull(input.Resolution_Notes,"") != "" && input.Resolved_On >=
input.Raised_On)
{
    student = Students[ID == input.Student];
    sendmail
    [
        from :zoho.adminuserid
        to :student.Email
        subject :"Complaint No. " + input.Complaint_No + "
resolved"
```

```
        message :"Dear " + student.Full_Name + ",<br><br>Your " +
input.Category + " complaint has been resolved on " +
input.Resolved_On + ".<br>Notes: " + input.Resolution_Notes +
"<br><br>Regards,<br>Hostel Office"
    ]
}
```

C.14 DS-13 Complaint_Status_Changed

```
// form: Complaints
// event: on_user_input
// field: Complaint_Status
// record_event: add_or_edit
// label: Complaint - show resolution fields
// ---
if(input.Complaint_Status == "Resolved")
{
    show Resolved_On;
    show Resolution_Notes;
    if(input.Resolved_On == null)
    {
        input.Resolved_On = zoho.currentdate;
    }
}
else
{
    input.Resolved_On = null;
    hide Resolved_On;
    hide Resolution_Notes;
}
```

C.15 DS-14 Complaint_Student_Selected

```
// form: Complaints
// event: on_user_input
// field: Student
// record_event: add_or_edit
// label: Complaint - fetch room
// ---
// Fetches the student's current room so it isn't entered again.
input.Room = null;
if(input.Student != null)
{
    student = Students[ID == input.Student];
    input.Room = student.Current_Room;
}
```

C.16 DS-15 Complaint_Validate

```
// form: Complaints
// event: on_validate
// record_event: add_or_edit
// label: Complaint - validate
// ---
error = "";
if(input.Room == null)
{
```

```
    student = Students[ID == input.Student];
    input.Room = student.Current_Room;
}
if(input.Raised_On > zoho.currentdate)
{
    error = "The raised date cannot be in the future.";
}
else if(input.Complaint_Status == "Resolved")
{
    if(input.Resolved_On == null ||
ifnull(input.Resolution_Notes,"") == "")
    {
        error = "Enter the resolved date and the resolution
notes.";
    }
    else if(input.Resolved_On < input.Raised_On)
    {
        error = "The resolved date cannot be before the raised
date.";
    }
}
// Duplicate check: one open complaint per room and category.
if(error == "" && input.Complaint_Status != "Resolved" &&
input.Room != null)
{
    for each c in Complaints[Room == input.Room && Category ==
input.Category && Complaint_Status != "Resolved"]
    {
        if(c.ID != input.ID)
        {
            error = "A " + input.Category + " complaint for this
room is already open (No. " + c.Complaint_No + ").";
        }
    }
}
if(error != "")
{
    alert error;
    cancel submit;
}
```

C.17 DS-16 Fee_Form_Load

```
// form: Fee_Payments
// event: on_load
// record_event: add_or_edit
// label: Fee - set defaults
// ---
// Defaults for a new bill: this month, due in 10 days.
if(input.Payment_Status == null)
{
    input.Payment_Status = "Pending";
    input.Billing_Month = zoho.currentdate.toString("MMMM");
    input.Billing_Year = zoho.currentdate.getYear();
    input.Due_Date = zoho.currentdate.addDay(10);
}
disable Room;
disable Total_Amount;
if(input.Payment_Status != "Paid")
```

```
{
    hide Payment_Date;
    hide Payment_Mode;
    hide Transaction_Reference;
}
else if(input.Payment_Mode == "Cash")
{
    hide Transaction_Reference;
}
```

C.18 DS-17 Fee_Mode_Changed

```
// form: Fee_Payments
// event: on_user_input
// field: Payment_Mode
// record_event: add_or_edit
// label: Fee - show transaction reference
// ---
// Cash needs no reference; UPI, card and bank transfers do.
if(input.Payment_Mode == null || input.Payment_Mode == "Cash")
{
    input.Transaction_Reference = "";
    hide Transaction_Reference;
}
else
{
    show Transaction_Reference;
}
```

C.19 DS-18 Fee_Receipt

```
// form: Fee_Payments
// event: on_success
// record_event: add_or_edit
// label: Fee - email receipt
// ---
// Notification: email a receipt once the fee is paid.
if(input.Payment_Status == "Paid")
{
    student = Students[ID == input.Student];
    sendmail
    [
        from :zoho.adminuserid
        to :student.Email
        subject :"Hostel fee receipt " + input.Receipt_No + " - "
+ input.Billing_Month + " " + input.Billing_Year
        message :"Dear " + student.Full_Name + ",<br><br>We
received Rs. " + input.Total_Amount + " on " + input.Payment_Date
+ " by " + input.Payment_Mode + " for " + input.Billing_Month + "
" + input.Billing_Year + ".<br>Receipt number: " +
input.Receipt_No + "<br><br>Regards,<br>Hostel Office"
    ]
}
```

C.20 DS-19 Fee_Status_Changed

```
// form: Fee_Payments
```

```
// event: on_user_input
// field: Payment_Status
// record_event: add_or_edit
// label: Fee - show payment details
// ---
// Conditional behaviour: payment details only matter once the fee
is paid.
if(input.Payment_Status == "Paid")
{
    show Payment_Date;
    show Payment_Mode;
    if(input.Payment_Date == null)
    {
        input.Payment_Date = zoho.currentdate;
    }
    if(input.Payment_Mode != null && input.Payment_Mode != "Cash")
    {
        show Transaction_Reference;
    }
}
else
{
    input.Payment_Date = null;
    input.Payment_Mode = null;
    input.Transaction_Reference = "";
    hide Payment_Date;
    hide Payment_Mode;
    hide Transaction_Reference;
}
```

C.21 DS-20 Fee_Student_Selected

```
// form: Fee_Payments
// event: on_user_input
// field: Student
// record_event: add_or_edit
// label: Fee - fetch room and rent
// ---
// Fetches the student's room and starts the bill with the monthly
rent.
input.Room = null;
if(input.Student != null)
{
    student = Students[ID == input.Student];
    input.Room = student.Current_Room;
    alloc = Room_Allocations[Student == input.Student &&
Allocation_Status == "Active"];
    if(alloc.count() > 0)
    {
        row = Fee_Payments.Fee_Items();
        row.Fee_Type = "Room Rent";
        row.Amount = alloc.Monthly_Rent;
        rows = Collection();
        rows.insert(row);
        input.Fee_Items.insert(rows);
    }
}
```

C.22 DS-21 Fee_Validate

```
// form: Fee_Payments
// event: on_validate
// record_event: add_or_edit
// label: Fee - validate and calculate total
// ---
error = "";
total = 0.0;
items = 0;
// Calculates the total of the fee items in the subform.
for each row in input.Fee_Items
{
    items = items + 1;
    if(row.Amount == null || row.Amount <= 0)
    {
        error = "Every fee item needs an amount greater than
zero.";
    }
    else
    {
        total = total + row.Amount;
    }
}
// Kept in a variable: Creator drops parentheses around an ||
inside an && when it saves a script.
bad_year = input.Billing_Year < 2020 || input.Billing_Year > 2100;
if(error == "" && items == 0)
{
    error = "Add at least one fee item.";
}
else if(error == "" && bad_year)
{
    error = "Enter a valid billing year.";
}
else if(error == "" && input.Payment_Status == "Paid")
{
    if(input.Payment_Date == null || input.Payment_Mode == null)
    {
        error = "Enter the payment date and payment mode for a
paid fee.";
    }
    else if(input.Payment_Date > zoho.currentdate)
    {
        error = "Payment date cannot be in the future.";
    }
    else if(input.Payment_Mode != "Cash" &&
ifnull(input.Transaction_Reference,"") == "")
    {
        error = "Enter the transaction reference for " +
input.Payment_Mode + " payments.";
    }
}
// Duplicate check: one bill per student per month.
if(error == "")
{
    for each f in Fee_Payments[Student == input.Student &&
Billing_Month == input.Billing_Month && Billing_Year ==
input.Billing_Year]
    {
```

```
        if(f.ID != input.ID)
        {
            error = "This student already has a fee record for " +
input.Billing_Month + " " + input.Billing_Year + ".";
        }
    }
}
if(error != "")
{
    alert error;
    cancel submit;
}
input.Total_Amount = total;
if(input.Room == null)
{
    student = Students[ID == input.Student];
    input.Room = student.Current_Room;
}
// Status update: an unpaid bill past its due date is overdue.
if(input.Payment_Status == "Pending" && input.Due_Date <
zoho.currentdate)
{
    input.Payment_Status = "Overdue";
}
```

C.23 DS-22 Leave_Dates_Changed

```
// form: Leave_Requests
// event: on_user_input
// field: To_Date
// record_event: add_or_edit
// label: Leave - count days
// ---
// Automatic calculation of the number of days, both dates
included.
if(input.From_Date != null && input.To_Date != null &&
input.To_Date >= input.From_Date)
{
    input.Number_of_Days =
input.From_Date.daysBetween(input.To_Date) + 1;
}
```

C.24 DS-23 Leave_Decision

```
// form: Leave_Requests
// event: on_validate
// record_event: edit
// label: Leave - approval decision email
// ---
// Approval workflow: when the warden approves or rejects, tell
the student.
// This runs while validating the edit, because that is where
old.* (the saved values) is
// available; it only sends once the decision is valid (a
rejection needs remarks).
// Kept in a variable: Creator drops parentheses around an ||
inside an && when it saves a script.
```

```
valid_decision = input.Approval_Status != "Rejected" ||
ifnull(input.Warden_Remarks,"") != "";
if(input.Approval_Status != old.Approval_Status &&
input.Approval_Status != "Pending" && valid_decision)
{
    student = Students[ID == input.Student];
    sendmail
    [
        from :zoho.adminuserid
        to :student.Email
        subject :"Leave request " +
input.Approval_Status.toLowerCase()
        message :"Dear " + student.Full_Name + ",<br><br>Your " +
input.Leave_Type + " leave from " + input.From_Date + " to " +
input.To_Date + " (" + input.Number_of_Days + " days) has been
<b>" + input.Approval_Status.toLowerCase() + "</b>.<br>Warden
remarks: " + ifnull(input.Warden_Remarks,"-") +
"<br><br>Regards,<br>Hostel Warden"
    ]
}
```

C.25 DS-24 Leave_Form_Load

```
// form: Leave_Requests
// event: on_load
// record_event: add
// label: Leave - new request defaults
// ---
input.Approval_Status = "Pending";
input.From_Date = zoho.currentdate.addDay(1);
input.To_Date = zoho.currentdate.addDay(1);
input.Number_of_Days = 1;
// Students only submit a request; the warden fills these when
reviewing it.
hide Approval_Status;
hide Warden_Remarks;
disable Number_of_Days;
```

C.26 DS-25 Leave_Review_Load

```
// form: Leave_Requests
// event: on_load
// record_event: edit
// label: Leave - warden review layout
// ---
// When the warden opens a request, only the decision can change.
disable Student;
disable Leave_Type;
disable From_Date;
disable To_Date;
disable Number_of_Days;
disable Reason;
```

C.27 DS-26 Leave_Validate

```
// form: Leave_Requests
// event: on_validate
```

```
// record_event: add_or_edit
// label: Leave - validate dates and overlap
// ---
error = "";
student = Students[ID == input.Student];
if(input.To_Date < input.From_Date)
{
    error = "To date cannot be before the from date.";
}
else if(input.From_Date.daysBetween(input.To_Date) + 1 > 30)
{
    error = "Leave cannot exceed 30 days. Please meet the
warden.";
}
else if(student.Student_Status != "Active")
{
    error = "Only active hostel residents can apply for leave.";
}
else if(input.Approval_Status == "Rejected" &&
ifnull(input.Warden_Remarks,"") == "")
{
    error = "Give a reason in Warden Remarks when rejecting a
request.";
}
// Duplicate check: requests of the same student must not overlap.
if(error == "" && input.Approval_Status != "Rejected")
{
    for each l in Leave_Requests[Student == input.Student &&
Approval_Status != "Rejected"]
    {
        if(l.ID != input.ID && l.From_Date <= input.To_Date &&
l.To_Date >= input.From_Date)
        {
            error = "This overlaps another leave request from " +
l.From_Date + " to " + l.To_Date + ".";
        }
    }
}
if(error != "")
{
    alert error;
    cancel submit;
}
input.Number_of_Days = input.From_Date.daysBetween(input.To_Date)
+ 1;
```

C.28 DS-27 Room_Form_Load

```
// form: Rooms
// event: on_load
// record_event: add_or_edit
// label: Room - set defaults
// ---
// Defaults for a new room.
if(input.Room_Status == null)
{
    input.Room_Status = "Available";
}
if(input.Occupied_Beds == null)
```

```
{
    input.Occupied_Beds = 0;
}
// Bed counts are kept up to date by the allocation workflows, so
nobody types them.
disable Occupied_Beds;
disable Available_Beds;
```

C.29 DS-28 Room_Type_Changed

```
// form: Rooms
// event: on_user_input
// field: Room_Type
// record_event: add_or_edit
// label: Room - capacity from room type
// ---
// Conditional field update: the room type decides the usual
number of beds.
if(input.Room_Type == "Single")
{
    input.Capacity = 1;
}
else if(input.Room_Type == "Double")
{
    input.Capacity = 2;
}
else if(input.Room_Type == "Triple")
{
    input.Capacity = 3;
}
```

C.30 DS-29 Room_Validate

```
// form: Rooms
// event: on_validate
// record_event: add_or_edit
// label: Room - validate and update status
// ---
error = "";
occupied = ifnull(input.Occupied_Beds,0);
if(input.Capacity < 1 || input.Capacity > 4)
{
    error = "Capacity must be between 1 and 4 beds.";
}
else if(input.Floor < 0 || input.Floor > 10)
{
    error = "Floor must be between 0 (ground floor) and 10.";
}
else if(input.Monthly_Rent < 1000 || input.Monthly_Rent > 50000)
{
    error = "Monthly rent must be between Rs. 1,000 and Rs.
50,000.";
}
else if(input.Capacity < occupied)
{
    error = "This room already has " + occupied + " students, so
its capacity cannot be lower than that.";
}
```

```
if(error != "")
{
    alert error;
    cancel submit;
}
// Automatic calculation and status update.
input.Occupied_Beds = occupied;
input.Available_Beds = input.Capacity - occupied;
if(input.Room_Status != "Under Maintenance")
{
    if(occupied >= input.Capacity)
    {
        input.Room_Status = "Full";
    }
    else
    {
        input.Room_Status = "Available";
    }
}
```

C.31 DS-30 Student_Form_Load

```
// form: Students
// event: on_load
// record_event: add_or_edit
// label: Student - set defaults
// ---
if(input.Student_Status == null)
{
    input.Student_Status = "Active";
}
// The current room is filled in by the allocation workflows.
disable Current_Room;
```

C.32 DS-31 Student_Validate

```
// form: Students
// event: on_validate
// record_event: add_or_edit
// label: Student - validate details
// ---
error = "";
phoneDigits = input.Phone.replaceAll("[^0-9]","");
guardianDigits = input.Guardian_Phone.replaceAll("[^0-9]","");
if(!input.Student_ID.startsWith("HS"))
{
    error = "Student ID must start with HS, for example
HS2026001.";
}
else if(input.Date_of_Birth >= zoho.currentdate)
{
    error = "Date of birth must be in the past.";
}
else if(input.Date_of_Birth.daysBetween(zoho.currentdate) < 16 *
365 || input.Date_of_Birth.daysBetween(zoho.currentdate) > 35 *
365)
{
```

```
    error = "A hostel resident must be between 16 and 35 years
old.";
}
else if(phoneDigits.length() < 10 || guardianDigits.length() < 10)
{
    error = "Enter valid 10-digit phone numbers for the student
and the guardian.";
}
else if(phoneDigits == guardianDigits)
{
    error = "The guardian's phone number must be different from
the student's.";
}
// Duplicate check: no two students share a phone number.
if(error == "")
{
    for each s in Students[Phone == input.Phone]
    {
        if(s.ID != input.ID)
        {
            error = "Phone number already registered for " +
s.Full_Name + " (" + s.Student_ID + ").";
        }
    }
}
if(error != "")
{
    alert error;
    cancel submit;
}
```

C.33 DS-32 Daily_Fee_Overdue_Check

```
// start: 2026-10-21 08:00:00
// frequency: daily
// time_zone: Asia/Kolkata
// label: Daily fee overdue check
// ---
// Scheduled workflow: every morning, mark unpaid bills past their
due date as overdue
// and remind the student of everything they owe.
for each fee in Fee_Payments[Payment_Status == "Pending" &&
Due_Date < zoho.currentdate]
{
    fee.Payment_Status = "Overdue";
    student = Students[ID == fee.Student];
    dues = thisapp.pendingDues(fee.Student);
    sendmail
    [
        from :zoho.adminuserid
        to :student.Email
        subject :"Hostel fee overdue - " + fee.Billing_Month + " "
+ fee.Billing_Year
        message :"Dear " + student.Full_Name + ",<br><br>Your
hostel fee of Rs. " + fee.Total_Amount + " for " +
fee.Billing_Month + " " + fee.Billing_Year + " was due on " +
fee.Due_Date + " and is now overdue.<br>Total pending dues: Rs. "
+ dues + "<br><br>Please pay at the hostel
office.<br><br>Regards,<br>Hostel Office"
    ]
}
```

APPENDIX D – USER MANUAL

D.1 Accessing the Application

Open the authenticated application URL shown in Project at a Glance and sign in with the assigned Zoho account. Use Dashboard for counters, charts, pending work and the Allocate Room, Record Fee, Apply Leave and Raise Complaint buttons. The left navigation opens each form and report. The screenshots show owner access; separate-role and native mobile login still require verification.

D.2 Instructions for Administrator and Warden

- Register a resident in Students, check Available Rooms, then use Room Allocations to link the resident to an available room. Verify Current Allocations and All Rooms after saving. For checkout, edit the allocation to Vacated and supply the vacate date.
- Use Leave Approvals to review a request and set Approved or Rejected; give a reason when rejecting. Use Complaints Board to track maintenance, and supply a resolution date and notes when resolving. Verify the changes in the corresponding list reports.

D.3 Instructions for Accountant and Student

- Accountant: open Fee Payments, select the student, review the fetched room and Fee Items, confirm the billing period and save. To record settlement, set Paid, enter the payment date and mode, and supply a reference for non-cash payments. Check Pending and Overdue Fees afterwards.
- Student: with an assigned Student profile, view Available Rooms, submit Leave Requests or Complaints, and track My Leave Requests or My Complaints. These instructions describe intended profile behaviour; a separate Student login must still verify access isolation.

APPENDIX E – EVALUATION RUBRIC

(For Examiner Use Only)

Student Name	Prajan Bala S N	Register No.	2116241501144
Project Title	Hostel Management System	Date	_____

Table E.1: Evaluation Rubric for the Individual Project

S.No.	Criterion	Indicators	Max. Marks	Marks Awarded
1	Problem Definition & Requirement Analysis	Clarity of problem, measurable objectives, complete FR/NFR, use cases	10	
2	System Design	Architecture, ER/data model, form specifications, workflow design, permission matrix	10	
3	Forms, Reports & Pages	Variety of field types, lookups/subforms, report types, UI Builder dashboards	15	
4	Deluge Scripting & Workflows	Correctness and depth of scripts; form, schedule, approval, Blueprint, batch workflows	15	
5	Integration, RBAC & AI	Zoho/external integrations, connections, roles & permissions, Zia/analytics	10	
6	Testing, Deployment, Security & Governance	Test coverage, environments, optimisation, security controls, audit	10	
7	Live Demonstration	End-to-end working of the application across roles; feature checklist verified	15	
8	Project Report	Adherence to template and format, clarity, figures, references	5	
9	Viva-Voce	Understanding of the platform, design decisions and low-code concepts	10	
	TOTAL		**100**	

Table E.2: Performance Level Descriptors

Level	% of Max. Marks	Descriptor
Excellent	90–100	Feature implemented completely, correctly and beyond the basic requirement; student explains design choices confidently.
Good	70–89	Feature implemented correctly with minor gaps; explanation adequate.
Satisfactory	50–69	Feature partially implemented or works only for the main path; limited explanation.
Needs Improvement	< 50	Feature missing, non-functional or not understood by the student.

Signature of Internal Examiner

Name: _____

Signature of External Examiner

Name: _____

APPENDIX F – VIVA-VOCE RECORD SHEET

(For Examiner Use Only)

Student Name	Prajan Bala S N	Register No.	2116241501144
Department / Section	B. Tech AI and ML	Date of Viva	_____
Application URL Verified	Examiner to verify	Demo Duration	_____ min

Table F.1: Viva-Voce Questions and Responses

Q.No.	Question / Topic Asked	Unit	Response (E / G / S / NI)	Remarks
1				
2				
3				
4				
5				
6				
7				
8				

E – Excellent G – Good S – Satisfactory NI – Needs Improvement

Table F.2: Marks Summary

Component	Max. Marks	Marks Awarded
Criteria 1–8 (Appendix E)	90	
Viva-Voce	10	
TOTAL	**100**	

Examiners' Remarks:

_____

_____

INTERNAL EXAMINER **EXTERNAL EXAMINER**

Name: _____ Name: _____